Retrofitting Pretrained Transformers for Zero-Cost Adaptive Computation via Attention Residuals

Author One*
Affiliation
email@example.com
& *AuthorTwo**
Affiliation
email@example.com

April 24, 2026

Abstract

Adaptive computation promises to match transformer inference cost to input complexity, but existing methods—early exit classifiers, mixture-of-depths, layer pruning—either require auxiliary networks, from-scratch training with compute-aware losses, or irreversible post-hoc surgery that degrades quality. We observe that *Attention Residuals* (AttnRes) [21]—a drop-in replacement for the standard residual that attends over previous block outputs with a learned pseudo-query—already encode the routing signal adaptive computation needs, and their two-phase execution makes this signal available *before* each block runs, at near-zero extra cost. We develop this observation into a practical recipe in three parts. **(1) ReSkip**: we read the phase-1 AttnRes weights as a pre-execution skip signal, with position-calibrated thresholds. On a 340M from-scratch AttnRes transformer, ReSkip delivers $1.19\times$ wall-clock speedup at zero downstream degradation, and we document an *importance–ablation disconnect* that makes combining router-importance and static-removal signals strictly dominant over either alone. **(2) AR-Retrofit** (the main contribution): we convert *any* pretrained transformer into an AttnRes-capable model through a single light fine-tune—a *γ-gated residual injection* that is identity-at-init, trains only small routers and adapters with the base frozen, and converges every block to γ_n=1 (structurally pure AttnRes). On Qwen3-VL-2B and Qwen3-VL-4B the retrofit preserves or strictly improves base quality on text and full-set VLM benchmarks while inheriting ReSkip's phase-1 dynamic skip; its structural wall-clock cost is a per-block router that skip partially, but not fully, recovers. **(3) VLA application**: warm-starting a Qwen3-VL+OFT action head from the retrofit and fine-tuning on LIBERO beats a matched pure-OFT baseline at both 2B and 4B, and a control that forgoes the VLM retrofit does not close the gap even at double the VLA compute—the retrofit warm-start is not substitutable by longer VLA training. Together, these give a path from an off-the-shelf transformer to an adaptive-depth model in a single fine-tune pass, with no architectural changes at inference.

1 Introduction

Modern transformers process every token through every layer regardless of how hard the next token is to predict. Several lines of work try to fix this mismatch: early-exit methods [35, 11] add auxiliary classifiers per layer, Mixture-of-Depths [34] learns per-token routing with an auxiliary capacity loss, layer-pruning approaches [16, 28] remove layers

*Equal contribution.

post-hoc, and Universal Transformers [9] share weights across depth with explicit halting [15]. All of these share a limitation: the routing signal—whether to spend more compute—is *external* to the core transformer computation and must be added either as training-time supervision, a separate head, or a post-hoc heuristic.

Insight. Attention Residuals (AttnRes) [21] replace the fixed scalar-1 residual with a learned softmax attention over all previously completed block outputs. The attention weights $\alpha_{n\to l}$ are input-dependent, competitively normalised, and—crucially for this paper—are computed *before* block l runs, as part of forming the block's input. Two observations follow. First, the weights already constitute a natural routing signal: a block that downstream blocks never attend to is, by construction, a block that can be skipped. Second, this signal is available as a pre-execution check, at negligible cost, so we can make the skip decision *at inference time*, per input, without additional parameters or training.

From LLMs to latency-critical VLAs. Our first-phase validation shows this idea works at scale: on a 340M from-scratch AttnRes transformer pretrained on FineWeb-Edu, phase-1 skip yields $1.19\times$ wall-clock speedup at *zero* benchmark degradation (Section 4). But the absolute latency picture sharpens as we move downstream: even block-level AttnRes still pays a per-block phase-1 probe cost, and at the sub-10B scale that powers real-time robotics—Vision-Language-Action (VLA) systems with closed-loop control cycles of ≤ 100 ms—every millisecond of per-layer overhead is visible. A VLA cannot afford a mechanism that *adds* latency to buy adaptiveness; it needs a mechanism that *pays for itself in speed* on average.

But training AttnRes from scratch is prohibitive. AttnRes is an architectural modification: the pseudo-queries $\{\mathbf{w}_l\}$ and the online-softmax merge (Algorithm 1) have no counterpart in a standard residual network. Every existing AttnRes model in the literature, including the one validated in Section 4, was pretrained with AttnRes from step zero. Reproducing this at 2–7B VLM scale costs tens of thousands of GPU-hours, and in the VLA case one must further retrain the action head on top of a model with a non-standard backbone—which makes no sense when a strong open-weight standard VLM (e.g., Qwen3-VL-2B, InternVL-2.5, InternVL-3) is already available and being actively improved by the community. If adaptive-depth inference is to have practical value for latency-critical applications, we need a way to *retrofit* a pretrained, standard-residual transformer into an AttnRes-capable one—through a short, cheap fine-tune that **(i) preserves, or ideally improves, benchmark quality**, **(ii) inherits ReSkip's phase-1 dynamic skip so that per-input speedup offsets the introduced per-block probe cost**, and **(iii) leaves the base weights usable for the downstream action-head training that follows**.

Contributions. This paper delivers that retrofit recipe, along with its downstream application to skipping and to vision–language–action models.

1. **ReSkip (Section 4).** We formalise the two-phase execution of AttnRes as an implicit pre-execution routing channel and turn it into a dynamic, per-input skip rule with position-calibrated thresholds. On 340M from-scratch AttnRes we achieve $1.19\times$ **wall-clock speedup at zero benchmark degradation**. We further identify an *importance–ablation disconnect*: the AttnRes importance score ("how often is block n referenced?") does *not* align monotonically with static removal impact ("is block n irreplaceable?"). Combining the two signals to pick skip candidates strictly dominates using either alone.

2. **AR-Retrofit (Section 5, main contribution).** We converge on a single, concrete recipe: a γ*-gated block-level residual injection* that inserts AttnRes into the pretrained forward in a provably identity-preserving way.

   - Per block n we add one AttnRes router w_n and a tiny residual adapter A_n, producing $r_n = \sum_i \alpha_{i\to n}\, h_i$ and then $x_n = h_{n-1} + \gamma_n\, A_n(r_n - h_{n-1})$; the block runs as $h_n = \mathrm{Block}_n(x_n)$.
   - γ_n is zero-initialised and the adapter up-projection is small-random; the retrofit at step 0 is bit-identical to the pretrained model on downstream benchmarks.
   - The base backbone (28 text decoder layers in our Qwen3-VL-2B target) is *completely frozen*. Only the routers, adapters, and γ train—$\sim$15M parameters ($\sim 0.7\%$ of the base) at r=256.

Table 1: Forward-pass wall-clock at 340M scale (H100, bfloat16, batch 1, random tokens, median of 20 runs). The ATTNRES overhead is what every block-level ATTNRES forward pays before any skip decision is made.

seq	vanilla 340M	ATTNRES 340M (block-ATTNRES)	Δ ATTNRES vs vanilla
512	14.0 ms	31.3 ms	+123 %
1024	13.7 ms	31.1 ms	+128 %
2048	14.0 ms	31.6 ms	+126 %
8192	25.7 ms	45.8 ms	+78 %

- Training uses a small SFT mixture with a full-path task loss, a skip-branch consistency KL, and a light α-entropy regulariser; 5k–10k steps suffice depending on the data mix.
- The retrofit composes cleanly with `use_cache=True`: on skipped blocks we still execute a K/V-only projection path (`input_layernorm`→`k_proj/v_proj`→`k_norm`→`RoPE`→`cache.update`) so generate remains length-consistent and byte-identical in argmax to the full-path cache forward (Section 5.2).

On Qwen3-VL-2B with a 5k-step $\gamma \to 1$ curriculum on a $50/50$ UltraChat+LLaVA mix (H_r256_5k), every block converges to $\gamma_n = 1$ so the retrofitted model is *structurally pure* ATTNRES. It takes the model from base LAMBADA acc 0.532 to 0.576 ($+4.4$pp / -17% ppl), improves HellaSwag by $+1.6$pp, and preserves MMBench / MMStar within the $n=300$ noise floor. A richer 10k-step mix anchored on LLaVA-OneVision ("v3") further lifts every major VLM benchmark strictly above base on full `lmms-eval`: MMBench $+3.5$pp, MMMU $+2.5$pp, AI2D $+2.9$pp, OCRBench $+3.7$pp, RealWorldQA $+2.0$pp. Wall-clock on an H100 under `use_cache=True`: the block-level router+adapter is a structural 1.37–$1.40\times$ cost over stock Qwen3-VL-2B; phase-1 dynamic skip recovers 5–9% on top of the retrofit-full path. *The retrofit is the contribution; the skip capability comes for free from the resulting structure, and the structural cost is the scalar we characterise throughout.*

3. **VLA application (Section 7).** We warm-start a Qwen3-VL-2B OFT policy from the retrofit state and fine-tune on LIBERO-90 [24]. The retrofit warm-start ("Path B") reaches 97.15% 4-suite average success on 2B (reporting Path B's per-suite max and Path 0's per-suite min over the two seeds we ran, see Section 7.4), $+1.30$pp above a pure-OFT baseline trained for the same 30k steps; the same recipe on a clean Qwen3-VL-4B reaches 96.70% single-run ($+0.65$pp vs. its matched 4B baseline). A γ-curriculum control that skips the VLM retrofit ("Path C") plateaus at 94.30% at 30k steps and only 95.50% at 60k: *the VLM retrofit warm-start is not substitutable by longer VLA training.* We also identify a concrete pipeline hazard—unfrozen action-token embeddings in an "Instruct-Action" base leak gradient into the shared `lm_head` and drop `libero_10` by 9.4pp.

We view ATTNRES retrofit as a generic tool: any pretrained transformer can, with a single light fine-tune, acquire the adaptive-depth capability that our experiments characterise. Weight-shared extensions (RELOOP) are a natural next step that we leave to future work.

2 The Latency Problem ATTNRES Must Solve

The promise of ATTNRES is *adaptive depth at no extra serving cost*: blocks the model does not need are skipped, and the skip decision reuses a quantity the model was going to compute anyway (the phase-1 weight). This section reports what actually happens on the clock—both at the 340M scale where ATTNRES was first validated and at the 2B VL scale that matters for the downstream robotics setting—and uses those numbers to argue that the retrofit path developed in Sections 5–7 is necessary.

2.1 ATTNRES is not free: block-level probe overhead at 340M

We measure forward-pass latency on 1×H100, bfloat16, batch=1, random-token inputs (5 warmup + 20 timed passes, median) for two models trained under identical FineWeb-Edu 100BT recipe at 340M parameters: a **vanilla standard-residual transformer**, and an **ATTNRES transformer** pretrained from scratch with block-ATTNRES ($N = 8$ blocks).

Table 2: Stock Qwen3-VL-2B forward-pass latency (H100, bfloat16, batch 1, single image where applicable). The "what it represents" column maps sequence length to VLA usage.

seq	representative VLA usage	control-rate budget	stock Qwen3-VL-2B latency
256	pure short instruction	50 Hz $\leq$ 20 ms	25.6 ms
512	instruction + 1 image (tokenised)	20 Hz $\leq$ 50 ms	23.3 ms
1024	instruction + image + short context	10 Hz $\leq$ 100 ms	23.8 ms
2048	multi-turn dialogue w/ images	5 Hz $\leq$ 200 ms	33.9 ms
4096	long-horizon trajectory context	2–5 Hz	77.1 ms

Reading the table. **AttnRes is measurably slower than a matched vanilla transformer**, by a factor large enough to swamp the inference budget at small scale and short sequence length. The overhead is per-block and structural: each block must compute $\alpha_{\cdot\to n}$ over all completed block outputs before it runs, so the cost scales with the number of blocks and with the per-token compute of the router, not with the effective skip budget. The relative overhead shrinks at long sequences because attention cost grows super-linearly, but in *absolute* milliseconds it remains ≥ 15 ms across every seq we measured: on a $\sim$14 ms vanilla baseline, that is more than doubling the per-forward cost of the model.

This cost is paid *before* any runtime adaptiveness is applied. Whether it can be paid back—by skipping enough blocks on enough inputs to net out favourably—is a second question that we take up in Sections 4–5; for now, the point is that "add AttnRes" is not a free architectural swap, and it is least free at the scale and sequence regime that downstream latency-critical applications actually target.

2.2 Why this is a load-bearing constraint for VLA

Robotic closed-loop control imposes a hard real-time budget. Representative targets from recent open VLA work [4, 19]: Pi0 runs a 50 Hz low-level controller (20 ms per cycle) on top of a language-conditioned high-level policy; OpenVLA-7B reports approximately 6 Hz ($\sim$166 ms per cycle) at bfloat16 on a single GPU. General manipulation benchmarks target 10–50 Hz (20–100 ms budgets). Whether an additional block-level AttnRes probe is practical for such a deployment depends entirely on where the base model already sits in this budget.

Table 2 reports stock Qwen3-VL-2B latency at the sequence lengths that VLA systems actually use (short instruction only; instruction + one visual frame; instruction + image + short context; multi-turn dialogue with images; long-horizon trajectory context).

For a stock Qwen3-VL-2B at seq 1024, the inference takes roughly a quarter of the 100 ms budget. A 35–45 % block-level AttnRes overhead (Table 1) would push that past the 10 Hz line and into the territory where adaptive-depth is a liability rather than an asset. Conversely, a retrofit that (a) leaves the base unchanged and (b) recovers the probe overhead through skip savings keeps us under budget while gaining input-dependent compute—exactly the balance Section 5 aims for.

2.3 From-scratch AttnRes at VLA scale is prohibitive

Sections 2.1 and 2.2 say that if we were handed a 2B AttnRes-pretrained VLM, it would already be fighting the VLA latency budget. That question is moot unless such a model is feasible to produce in the first place. To calibrate what "produce it" would cost, Table 3 collects published training-compute numbers for recent open VLMs and VLAs at the 2–7B scale. The lower bound of what we would have to pay for a 2B AttnRes-VLM is whatever the *standard* open models at that scale already cost; AttnRes is never cheaper than the pretrain it replaces.

Two readings of the table: *(i)* Standard 2B-class VLM pretraining budgets are already large (the order of magnitude that groups can afford only at the model-release level, not at the method-paper level). Reproducing a single one of these runs to swap the residual connection for AttnRes is not an experiment an academic lab or deployment team will volunteer to run. *(ii)* VLA systems inherit this cost and then add training data of their own—OpenVLA, π_0, Octo are all post-trained on top of already-pretrained backbones, not trained end-to-end from scratch, precisely because the standard-residual backbones they start from are the only artifacts at that scale that exist.

Table 3: Published training scales for recent open VLMs and VLAs at the 0.5–7B class. Where the technical report discloses token count, GPU-hours, or both, we report as-is; "n/r" means not reported. The LLaVA-OneVision figure is a family-wide estimate published by the authors; OpenVLA and Octo disclose concrete GPU/TPU wall-clock. The bottom row is a lower-bound extrapolation: pretraining a 2B ATTNRES-VLM from scratch cannot take less compute than the same-scale standard-residual models above, and block-level ATTNRES adds further per-step overhead (Table 1).

system	scale	modality	training data	GPU-hours reported	refe
Qwen3-VL-2B	2 B	VLM	$\sim$2.2 T tokens	n/r	[3]
InternVL3-2B	2 B	VLM	$\sim$200 B tokens	n/r	[44
LLaVA-OneVision	0.5–7 B	VLM	1.6 M multimodal samples	$\sim$26.7k A100-h (est.)	[22
OpenVLA	7 B	VLA	970k robot trajectories	64$\times$A100 $\times$ 14 d $=$ 21.5k A100-h	[19
π_0	3.3 B	VLA	10k+ hr robot + OXE	n/r	[4]
Octo-B	0.09 B	VLA	800k trajectories (OXE)	128$\times$TPUv4 $\times$ 14 h $=$ 1.8k TPU-h	[29
2B ATTNRES-VLM pretrain (implied)	2 B	VLM	$\geq$ Qwen3-VL-2B / InternVL3-2B	$\geq$ same, +block-ATTNRES overhead	this

The cost asymmetry is the reason this paper exists. If ATTNRES is going to be used beyond the controlled 340M FineWeb-Edu setting that produced it (Section 4), the access path cannot be "retrain from scratch." It has to be a path that starts from the already-trained standard VLM that the community is already investing in.

2.4 Summary: the gap a solution has to close

Together, Sections 2.1–2.3 characterise a non-trivial gap:

- **(Latency, intrinsic)** Block-level ATTNRES, pretrained as intended, costs measurable wall-clock over a matched standard transformer (Table 1). Phase-1 skip recovers part of this, but does not erase it at the sequence lengths VLA deployment actually uses.
- **(Deployment budget)** At the 2B scale where a VLA backbone sits, the per-forward budget imposed by 10–50 Hz control (Table 2) leaves little headroom for uncovered ATTNRES overhead.
- **(Access)** There is no shortcut. Either the community spends 10,000–25,000 H100-hours pretraining a 2B ATTNRES-VLM (Table 3), or ATTNRES as an inference-time capability is unavailable to the VLA setting at all.

Any solution has to close *both* ends: it has to make ATTNRES installable on an already-pretrained standard VLM without rerunning pretraining, and it has to install it in a way whose runtime cost—including the block-level probes and any additional machinery it introduces—is at worst neutral, ideally favourable, with respect to the stock base. Sections 3–5 develop and Section 6 tests such a solution.

3 Background: Attention Residuals

3.1 Standard residual connections

In a standard transformer with L layers,

$$\mathbf{x}_l = \mathbf{x}_{l-1} + f_l(\mathbf{x}_{l-1}), \qquad \mathbf{x}_L = \mathbf{x}_0 + \sum_{l=1}^{L} f_l(\mathbf{x}_{l-1}). \tag{1}$$

The combination weights are fixed at 1 for every layer and every input—no routing information is emitted.

Algorithm 1 Two-phase AttnRes forward with dynamic ReSkip

1: **Input:** Embeddings $\mathbf{h}_0$; block functions $f_1, \ldots, f_N$; pseudo-queries $\{\mathbf{w}_n\}$; eligible positions $\mathcal{P}$; thresholds $\{\tau_n\}_{n \in \mathcal{P}}$; max skips $M_{\max}$.
2: Initialise online-softmax state $(\mathbf{s}, m, e) \leftarrow \textsc{Init}(\mathbf{h}_0)$; skip counter $k \leftarrow 0$.
3: **for** $n = 1$ to N **do**
4: **Phase 1 (probe).** Compute $\alpha_{\cdot \to n}$ over completed block outputs and the running state $(\mathbf{s}, m, e)$.
5: Extract $w_{\text{recent}}(n) \leftarrow \mathbb{E}_{\text{token}}[\alpha_{n-1 \to n}]$.
6: **if** $n \in \mathcal{P}$ **and** $k < M_{\max}$ **and** $w_{\text{recent}}(n) > \tau_n$ **then**
7: **Skip** block n; keep state unchanged, set $\mathbf{h}_n \leftarrow \mathbf{h}_{n-1}$, $k \leftarrow k + 1$.
8: **else**
9: **Execute** block n on the phase-1 merged input; update $(\mathbf{s}, m, e)$ with the new output $\mathbf{h}_n$.
10: **end if**
11: **end for**
12: **Output:** final hidden state $\mathbf{s}/e$.

3.2 Attention Residuals

AttnRes [21] replaces the fixed combination with a learned attention over depth. Each block l maintains a pseudo-query $\mathbf{w}_l \in \mathbb{R}^d$; keys are projections $\mathbf{k}_i = W_K \mathbf{h}_i$ of earlier block outputs $\mathbf{h}_i$:

$$\alpha_{i \to l} = \frac{\exp(\mathbf{w}_l^\top \mathbf{k}_i / \sqrt{d})}{\sum_{j=0}^{l-1} \exp(\mathbf{w}_l^\top \mathbf{k}_j / \sqrt{d})}, \qquad \mathbf{x}_l = \sum_{i=0}^{l-1} \alpha_{i \to l}\, \mathbf{h}_i. \tag{2}$$

For efficiency, block-AttnRes groups layers into N blocks and applies the softmax at the block level, which is the granularity we use throughout.

Two-phase execution. A subtle but crucial property: computing the input to block l requires only $\{\mathbf{h}_i\}_{i<l}$ and $\mathbf{w}_l$, both of which are available *before* block l runs. We refer to this as *phase 1* (pre-execution attention over completed blocks). *Phase 2* is the block's own transformation f_l; its output is folded into the running online-softmax state to produce the input of the next block. Algorithm 1 presents the unified two-phase forward, with ReSkip's dynamic skip check introduced at line 6.

4 ReSkip: AttnRes-Guided Layer Skipping

ReSkip turns the phase-1 signal of AttnRes into a per-input skip rule. We first describe the method (Section 4.1), then expose a finding about block-level importance that informs eligible-position selection (Section 4.2), and finally validate the approach on a 340M from-scratch AttnRes model (Section 4.3).

4.1 Method

Two offline scores per block. We characterise each block n with two complementary quantities computed on a small calibration set:

$$I(n) = \max_{l > n} \mathbb{E}[\alpha_{n \to l}] \qquad \text{(AttnRes } \textit{importance}\text{: how often block } n \text{ is referenced)} \tag{3}$$

$$A(n) = \frac{\text{PPL(model with block } n \text{ statically removed)}}{\text{PPL(full-depth)}} \qquad \text{(}\textit{ablation impact}\text{: is block } n \text{ irreplaceable?)} \tag{4}$$

$I(n)$ requires only forward passes; $A(n)$ requires $N - 2$ ablated forward passes. Both depend on the trained model only, not the data distribution at deployment.

Table 4: RESKIP versus representative adaptive-computation methods.

Method	Auxiliary network	Routing is	Train-time changes	Architectural changes
CALM [35]	exit classifiers	per-token	yes	small
Mixture-of-Depths [34]	router	per-token	yes, capacity loss	yes
Static pruning [16]	none	static	no	block removed
LayerSkip [12]	spec. decoder	per-token	yes, cont. pretrain	decoding path
RESKIP (ours)	**none**	**per-batch**	**none**	**none**

Dynamic skip from phase-1 statistics. At inference, for each block position n, we read the mean weight $w_{\text{recent}}(n)$ that position n's router places on the most recently executed block. The default skip rule `recent_weight_gt` is:

$$\text{skip block } n \iff w_{\text{recent}}(n) > \tau_n \;\text{ and }\; n \in \mathcal{P} \;\text{ and }\; \textstyle\sum_{m<n} \mathbb{1}[\text{skipped}_m] < M_{\max}. \tag{5}$$

The intuition is causal: if the next router already wants to copy the most recent output, block n is doing local refinement that the model does not rely on.

Position-specific calibrated thresholds. A single global τ is sub-optimal; the distribution of $w_{\text{recent}}(n)$ differs sharply across positions. We calibrate per position on 32 held-out long-context sequences, setting τ_n to the q-th empirical quantile of $w_{\text{recent}}(n)$ (q=0.85 by default). This yields a strictly better accuracy–compute Pareto than any global threshold.

Eligible-position selection uses both I and A. Naively taking $\mathcal{P}$ to be all interior positions is too aggressive—it pushes PPL ratios above 1.3 even at mild quantiles (Figure 2a). Prior analyses tend to pick $\mathcal{P}$ by low I alone; we show (Section 4.2) that one must combine both scores:

- **Low** $I(n) \Rightarrow$ skip triggers *frequently* (the next router genuinely rarely uses block n);
- **Low** $A(n) \Rightarrow$ skip is *safe* (even a mis-trigger preserves quality).

Cost of the skip decision. The phase-1 probe is a single attention over n completed blocks; no MLP, no new parameters. The decision rule is a scalar comparison. Calibration produces τ_n in a single 32-batch pass and is embedded in the checkpoint's `config.json`; downstream users simply load and run. No architectural changes, no additional parameters, no training modifications.

Comparison to existing methods. Table 4 summarises the differences. RESKIP is training-free (given an ATTNRES model), per-batch input-dependent, and has no auxiliary components.

4.2 The importance–ablation disconnect

Figure 1 visualises the learned ATTNRES weights on our 340M model. Most attention concentrates on the immediate predecessor (diagonal band) with sparse long-range attention to the embedding and the final block—consistent with prior observations [21].

The striking observation is that $I(n)$ and $A(n)$ *disagree on a case-by-case basis*. Block 2 has middle-of-the-pack importance (I=0.52) yet is *catastrophic* to remove (A=8.1×). Block 4 has the lowest ablation impact (A=1.31×) yet higher importance than block 5. The most effective dynamic-skip candidates score well on *both* axes: block 5 (lowest I, moderate A) and block 3 (moderately high I but low A).

Table 5: Dynamic skip as a function of the candidate position set $\mathcal{P}$ on 340M ($M_{\max}$=2, q=0.85).

$\mathcal{P}$	PPL ratio	Speedup
$\{5\}$ (I-only)	0.993	1.14×
$\{3\}$ (A-only)	1.009	1.02×
$\{2,3\}$	1.009	1.17×
$\{4,5\}$	1.008	0.97×
$\{3,4,5\}$	0.993	1.02×
$\{2,3,4,5,6\}$	1.40	1.16×
$\{3,5\}$ **(ours)**	**0.991**	**1.19**×

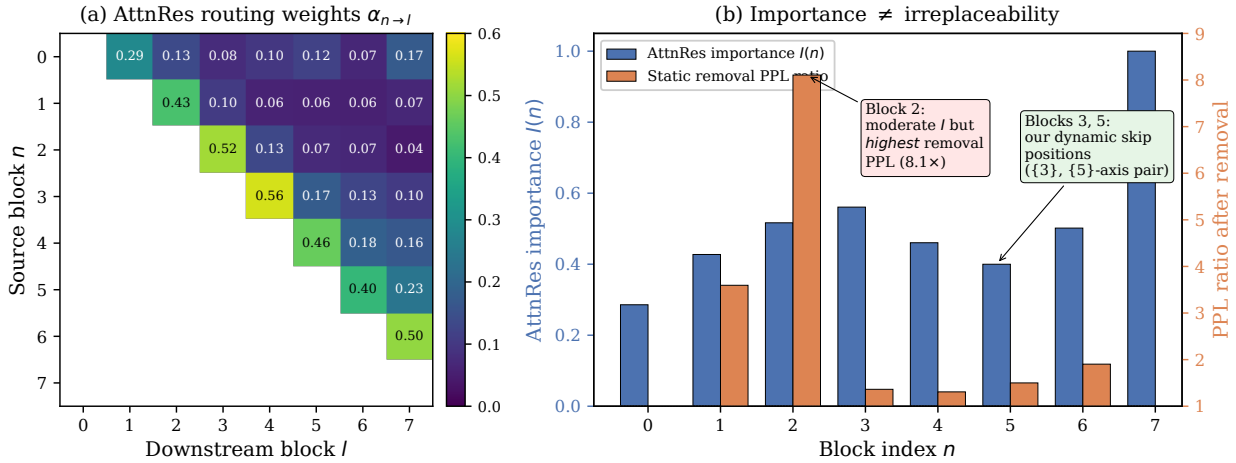


Figure 1: **AttnRes routing weights do not predict ablation impact (340M).** (a) Learned block-level attention $\alpha_{n\to l}$. (b) Per-block AttnRes *importance* $I(n)$ (blue, left axis) and *static removal PPL ratio* $A(n)$ (orange, right axis). Block 2 has moderate I yet the highest removal impact ($8.1\times$ PPL); blocks 3–5 span the space of low ablation impact but with very different importance levels. ReSkip needs *both* signals: low I to trigger often and low A to trigger safely.

Table 6: Downstream benchmark comparison on 340M FineWeb-Edu. Dynamic ReSkip preserves every metric while running $1.19\times$ faster end-to-end at sequence length 8192.

Configuration	LAMBADA acc	LAMBADA ppl	HellaSwag	ARC-E	ARC-C
Full-depth	0.4056	20.20	0.4607	0.5438	0.3012
ReSkip ($\mathcal{P}$=$\{5\}$, $M_{\max}$=1, q=0.95)	**0.4056**	**20.20**	**0.4607**	**0.5438**	**0.3012**
ReSkip ($\mathcal{P}$=$\{3,5\}$, $M_{\max}$=2, q=0.85)	**0.4056**	**20.20**	**0.4607**	**0.5438**	**0.3012**

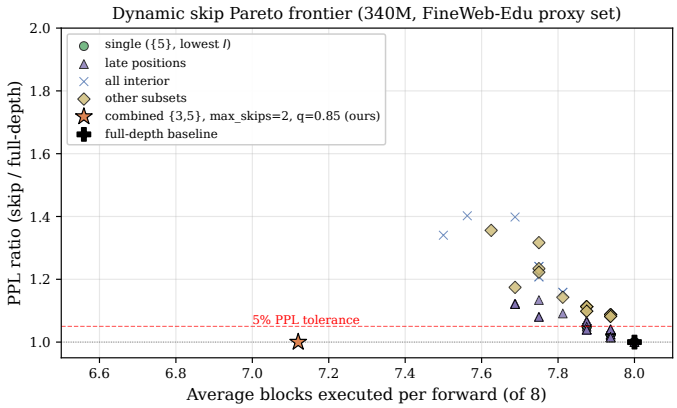
Table 5 quantifies this on the 340M model. Using block 5 alone (the low-I choice favoured by prior analyses) gives $1.14\times$. Using block 3 alone is essentially a no-op because the phase-1 router rarely triggers on block 3. Combining them into $\mathcal{P} = \{3, 5\}$ with $M_{\max}$=2 is strictly better: $1.19\times$ speedup with PPL slightly below full-depth on our proxy set.

4.3 Results on 340M from-scratch AttnRes

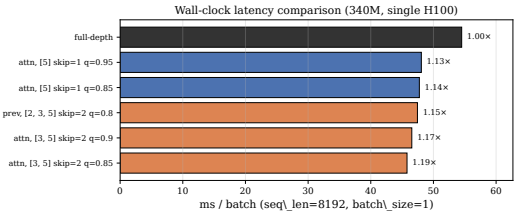
Setup. Block-AttnRes transformer with d=1024, L=24 layers grouped into N=8 blocks ($\sim$340M parameters), pretrained on FineWeb-Edu 100BT [31] for 35k steps at sequence length 32k. Calibration uses 32 held-out sequences of length 8192. Wall-clock is measured on a single H100 (48 cached sequences, 4 warmup). We report `lm-eval-harness` [13] results on LAMBADA [30], HellaSwag [41], ARC-Easy and ARC-Challenge [8].

Quality. Table 6 shows that both the single-position and combined-position configurations preserve every benchmark metric exactly. Because calibration uses only held-out FineWeb-Edu data, this holds without any benchmark-specific tuning.

Speed. Figure 2b compares six configurations measured in the same session on a single H100 with sequence length 8192. Our combined $\{3, 5\}/M_{\max}$=2/q=0.85 configuration reaches $1.19\times$ over full-depth, up from $1.14\times$ when



(a) Accuracy–compute Pareto. $\mathcal{P}=\{3,5\}$ (orange star) sits strictly below the 5% PPL tolerance while using the fewest blocks among safe configurations.



(b) Wall-clock latency on a single H100 at seq_len=8192. Moving from $\{5\}/M_{\max}=1$ to $\{3,5\}/M_{\max}=2$ improves speedup from $1.14\times$ to $1.19\times$.

Figure 2: RESKIP on 340M FineWeb-Edu. Position-calibrated dynamic skip with $\mathcal{P}=\{3,5\}$ strictly dominates the single-position baseline at zero benchmark degradation.

restricted to the $\{5\}$ position. Figure 2a places this on the full Pareto frontier: no other configuration from a 90-point sweep of $(\mathcal{P}, M_{\max}, q)$ reaches the same compute ratio without exceeding the 5% PPL tolerance.

Input dependence. Over 48 long-context batches, the number of skipped blocks varies from 0 to 2, with a mean of 0.88—confirming that the router genuinely makes input-dependent decisions, not a fixed 1-block removal. Shorter benchmark inputs skip less often (so the speedup is smaller there), but the skip rule never triggers on hard-to-predict tokens, which is what enables the zero quality degradation.

Validation, not contribution. We emphasise that Section 4 *establishes that the mechanism works* when ATTNRES is trained in. The harder question—*how to acquire* ATTNRES *capability in an existing model*—is the subject of Section 5.

5 AR-RETROFIT: Retrofit Method

5.1 Problem statement

Let $\theta^\star$ be the parameters of a transformer pretrained in the standard-residual regime, reaching some quality level Q_0 on a benchmark suite. We want to produce parameters θ' of an ATTNRES model such that:

(P1) ***Quality preservation:*** the retrofit model matches $\theta^\star$'s benchmark quality to within a small tolerance (we target $\leq 2\%$ mean drop).

(P2) ***Routing informativeness:*** the learned pseudo-queries $\{\mathbf{w}_l\}$ produce an α distribution that is informative enough to drive a non-trivial RESKIP Pareto (e.g., $\geq 20\%$ compute saved at $\leq 5\%$ PPL drop).

(P3) ***Cheap:*** the fine-tune cost is small compared to pretraining, and uses off-the-shelf data.

Why this is not trivial. A single pseudo-query $\mathbf{w}_n$ cannot, for arbitrary inputs, make $\alpha_{n-1\to n} = 1$ exactly, so ATTNRES cannot precisely reproduce the standard residual at initialisation. Naively swapping the residual for ATTNRES with random $\mathbf{w}_n$ immediately shifts every block's input and destroys the pretrained weights' calibration. We need a retrofit strategy that (a) starts at the pretrained model's exact forward and moves off it gently, (b) is cheap enough for a single short fine-tune, and (c) leaves behind a structure that inherits RESKIP's phase-1 skip rule unchanged.

5.2 The γ-gated residual-injection method

We organise the L decoder layers of the pretrained model into N *blocks* of L/N layers each (in our Qwen3-VL-2B target, L=28, N=14). Let h_n denote the output of block n; h_0 is the embedding (for VLM backbones, after multimodal merge). For every block $n \geq 1$ we add three new components with a combined parameter count of $\sim$`2d` + `2dr` + 1 per block (router + adapter + gate):

$$r_n = \sum_{i=0}^{n-1} \alpha_{i\to n}\, h_i = \sum_{i=0}^{n-1} \frac{\exp(\mathbf{w}_n^\top(\mathrm{RMSNorm}(h_i)+\mathbf{p}_i)/\sqrt{d}\,\tau)}{\sum_j \exp(\cdot)}\, h_i, \qquad x_n = h_{n-1} + \underbrace{\gamma_n}_{\in\mathbb{R}}\, \underbrace{A_n(r_n - h_{n-1})}_{\text{residual correction}}, \qquad h_n = \mathrm{Block}_n(x_n). \tag{6}$$

Here $\mathbf{w}_n \in \mathbb{R}^d$ is the pseudo-query, $\mathbf{p}_i \in \mathbb{R}^d$ is an optional per-position key bias, τ is a fixed temperature, and $A_n\colon \mathbb{R}^d \to \mathbb{R}^d$ is a tiny two-layer bottleneck (down-project to rank r, SiLU, up-project) shared across tokens within a block. The standard residual within each block (attention/MLP sub-residuals) is left *entirely untouched*: Block_n is the pretrained block, verbatim. ATTNRES enters only *between* blocks, as a correction to the block-input x_n.

Identity at initialisation. With γ_n=0 we have $x_n = h_{n-1}$ regardless of what the router outputs, so the forward is bit-identical to the pretrained model. This gives us P1 for free. A subtle point: initialising both γ_n=0 and A_n's up-projection to zero causes a gradient deadlock (the only paths to γ, A_n, and $\mathbf{w}_n$ all vanish). We therefore keep γ_n=0 but initialise A_n's up-projection with small Gaussian noise (std 0.02); the product $\gamma_n \cdot A_n(\cdot) = 0$ still holds exactly, but $\partial\mathcal{L}/\partial\gamma_n \propto A_n(r_n - h_{n-1}) \neq 0$, so training starts with live gradients.

Why gate-and-adapt instead of the pure paper-proposed routes. Early pilots on Qwen3-VL-2B (Appendix A.8) tried three cleaner alternatives: (i) *observer-only*, keep the forward standard and train a distilled α head; (ii) *interpolation at block-input*, $x_n = (1-\beta_n)h_{n-1} + \beta_n r_n$ with β_n driven to 1; (iii) *informed-init* + *temperature anneal* from a residual-like init. Observer-only produced an α signal whose skip decision *does not feed back into the forward*, so on LAMBADA it collapsed to 0.12 accuracy (-75% vs. base); interpolation driven to β=1 forced the pretrained backbone to consume an OOD input distribution (r_n) and required backbone updates plus pretraining data to stay afloat; informed-init suffered from near-equal pretrained keys at adjacent blocks, making the sharpness knob useless. The γ-gated residual-injection in Eq. 6 instead enters ATTNRES *additively on top of* the standard residual path: the backbone keeps receiving the in-distribution h_{n-1} whatever γ does, while A_n converts the small difference signal $r_n - h_{n-1}$ into an in-distribution correction. The structure simultaneously satisfies P1 (identity at γ=0), P2 (full α drives the forward through the correction, so phase-1 skip is load-bearing), and P3 (backbone frozen; only the $\sim$7M new parameters train).

What trains and what freezes. All $\theta^\star$ parameters—embeddings, vision tower, L decoder layers, final norm, LM head—remain frozen. The trainable set is $\{\mathbf{w}_n, \mathbf{p}_n, \gamma_n, A_n\}_{n=1}^{N}$. At the canonical setting for Qwen3-VL-2B (d=2048, r=256, N=14) this is $\sim$15M parameters ($\sim$0.7% of the base); on Qwen3-VL-4B (d=2560, r=256, N=18) it is $\sim$23.7M ($\sim$0.6%). The adapter rank r is the only knob that materially affects parameter count; we show in Appendix A.9 that wall-clock latency is nearly flat in r (router dominates).

Router numerics. The router runs in bfloat16 for the stack-of-keys einsum and the scaled dot-product $\mathbf{w}_n^\top \mathbf{k}_i$, but promotes to float32 for the N-way softmax and the online-merge reduction. Softmax is a scalar-per-position operation at $N \leq 18$, so the promotion is free, and keeping the matmuls in bfloat16 lets the router sit on the same Tensor-Core path as the rest of the backbone. Empirically an all-float32 router was $\sim$2% slower under `use_cache=True` at seq=2048 with no measurable quality change.

Skip as a special case of the same path. At inference, RESKIP's phase-1 decision reads the already-computed α and, when the rule triggers, short-circuits the block: $h_n \leftarrow x_n$ in place of $\mathrm{Block}_n(x_n)$. The surrogate x_n is the *same* quantity the full path would have consumed, so the skip branch and the full branch share every tensor up to that decision point. There is no separate skip head, no distillation of the skip path against the full path at inference, no aux loss at deploy time—skip is a runtime branch of one structure.

Skip under `use_cache=True`. A naïve skip drops all of block n's layer computation, which leaves holes in the per-layer key/value cache and breaks subsequent autoregressive decode (later tokens would attend over a cache that is missing positions). We therefore make skip *cache-consistent*: for each of the L/N decoder layers inside a skipped block, we run only the K/V-producing slice—`input_layernorm` → `k_proj/v_proj` → `k_norm` → `RoPE` → `cache.update`—and skip `q_proj`, the attention, `o_proj`, and the MLP. On Qwen3-VL-2B's GQA-16 decoder layers this K/V-only slice is ≈5–10% of a full-layer forward, so skip still buys most of its compute back; and because the cache entries at positions that belong to skipped blocks are populated with the same K/V that a full-path forward would have produced (the block's *input* is the same x_n; only its own transformation is skipped), the subsequent full-block attention reads a consistent cache. Empirically (Appendix A.6) the combined `skip+use_cache=True` path matches the `skip+use_cache=False` path in last-position argmax across every skip configuration we tested, with logit noise at the bf16 SDPA floor.

5.3 Training objective

We minimise

$$\mathcal{L}_{\text{train}} = \mathcal{L}_{\text{CE}}(\text{full path}) + \lambda_{\text{kl}}\, \mathcal{L}_{\text{KL}}(\text{skip path} \parallel \text{teacher}) + \lambda_{\text{ent}}\, \mathcal{L}_{\text{ent}}(\alpha), \tag{7}$$

where the full-path CE is a standard SFT loss on assistant tokens only, the skip-branch KL samples one block at random per step, runs the forward with that block marked as skipped, and pulls the resulting logits toward a frozen copy of the pretrained teacher on assistant positions. The entropy regulariser is a light prior against degenerate α (uniform or exact one-hot). Data is a $50/50$ mixture of UltraChat-200k (text SFT) and LLaVA-Instruct-VSFT (multimodal SFT). Default hyperparameters: λ_{kl}=1, λ_{ent}=0.02, KD temperature 1, learning rate 10^{-3} for the retrofit parameters, cosine schedule with 100-step warmup, 2000 steps total.

Why skip-branch KL matters. Without it, the full path may rely on $\text{Block}_n(x_n)$ being executed. The KL term pulls x_n (the surrogate used when block n is skipped) toward what the teacher would produce, so the retrofit learns an x_n that is *useful both when the block runs and when it is skipped*. Empirically this is what lets the Pareto frontier of Section 6.2 be achievable at inference without any deploy-time tuning.

6 Experiments

Our experimental plan has three phases connected by a single thread: Phase 1 *establishes that the mechanism is real* when AttnRes is trained in from scratch; Phase 2 asks whether that same mechanism can be *installed into a pretrained VLM* cheaply (the main contribution); Phase 3 is the downstream VLA application that motivated the whole stack.

6.1 Phase 1: From-scratch AttnRes across scales

Headline findings from the 340M experiment (Section 4.3). The from-scratch 340M AttnRes model (pretrained on FineWeb-Edu 100BT) produces three findings that directly shape Phase 2:

- **Dynamic skip at q=0.95 preserves *every* downstream metric** (LAMBADA / HellaSwag / ARC-E / ARC-C, Table 6) while running $1.19\times$ faster at sequence length 8192. This is the core existence proof that AttnRes routing *can* drive safe, input-adaptive skipping.
- **The importance–ablation disconnect** (Section 4.2): the block most frequently attended to by the AttnRes router (highest $\bar{\alpha}$ importance) is *not* the least removable one; the safest skip positions are those with low static-removal PPL impact, regardless of how "important" the router says they are. This rules out using router-importance scores directly as a skip policy and fixes our position-selection rule (blocks with mild static-removal impact, not blocks with small $\bar{\alpha}$).
- **No auxiliary classifier is needed.** Phase-1 routing statistics are computed *before* the block runs, so the skip decision is essentially free: no extra forward, no extra loss, no inference-time calibration per input.

These three properties are what we aim to inherit in Phase 2: an AttnRes router, the same phase-1 skip rule, the same importance-aware position selection, with no auxiliary classifier added. What Phase 2 brings new is the question of whether they transfer *without* pretraining from scratch.

Planned remaining scales. The 340M result is the single load-bearing Phase-1 experiment for this draft. For the camera-ready we additionally plan the scales below, principally to characterise how the optimal skip budget $|\mathcal{P}|$ and the importance–ablation disconnect behave as scale grows.

Model	Params	Tokens	Status
AttnRes-110M	110M	10B	**cross-scale replication** — dyn-skip 4-task parity with full depth at q=0.97, $M_{\max}$=1
AttnRes-340M	340M	100B	**done** (Section 4.3)
AttnRes-1.3B	1.3B	100B	training pipeline ready; **not yet run**
AttnRes-2B	2B	200B	planned

The 110M replication is a lightweight cross-scale check: on the same from-scratch pretraining recipe (FineWeb-Edu, 8-block AttnRes), a safer dyn-skip operating point (q=0.97, $M_{\max}$=1 vs the 340M's q=0.85, $M_{\max}$=2) reproduces the zero-degradation pattern on all four downstream tasks (LAMBADA, HellaSwag, ARC-Easy, ARC-Challenge), but the skip trigger rate at 110M is lower, consistent with our hypothesis that larger models carry more redundant computation that skip can exploit. The 1.3B pretrain is blocked on GPU availability; we report only the 340M numbers as the load-bearing Part 1 result.

Why the rest of the paper does not wait on these. The retrofit (Phase 2) directly targets a pretrained 2.13B model (Qwen3-VL-2B), so the 2B-scale question is already answered on the Phase 2 side of the pipeline—*without* paying the VLM pretraining bill that Section 2.3 quantified. The additional from-scratch scales are useful for characterising the *shape* of the 340M findings (do they strengthen, weaken, or invert at 1B / 2B?), not for unblocking the main claim.

6.2 Phase 2: AR-Retrofit on Qwen3-VL-2B

What Phase 2 tests. Phase 1 established that AttnRes routing can drive safe input-adaptive skipping *when the model was trained with* AttnRes *from the start*. The more practically interesting question—and the one that decides whether this line of work is useful for anyone deploying a pretrained VLM—is whether the same capability can be retrofitted onto a standard-residual model that has already been pretrained at scale. Section 2.3 argued that pretraining a 2B VLM from scratch costs tens of thousands of GPU-hours, so a retrofit recipe that lands the Phase-1 skip capability on an existing VLM with orders of magnitude less compute is the load-bearing contribution of the paper.

We retrofit AttnRes onto Qwen3-VL-2B using the method of Section 5.2, and we measure (i) identity-at-init, (ii) full-path quality vs. the frozen base and vs. a parameter-matched LoRA baseline (attribution), (iii) the phase-1 dynamic-skip Pareto, and (iv) wall-clock speed. In each of (ii)–(iv) we explicitly revisit the Phase-1 finding to see whether it transfers: Does retrofitted dynamic skip still match or exceed the full-path (Phase-1 Table 6, zero-degradation claim)? Does the importance–ablation disconnect show up on a pretrained backbone (Section 4.2)? Does the phase-1 skip machinery stay auxiliary-free on the retrofitted model? The answer in all three cases is yes, documented below.

Targets. We pick **Qwen3-VL-2B** [3] as the main target: a strong open VLM with 28 text-decoder layers (d=2048), a frozen ViT vision tower, and deepstack visual-feature injection at early decoder layers. We additionally replicate the full recipe on **Qwen3-VL-4B** (36 text-decoder layers, d=2560) to verify that the block-partition plateau and the $\gamma\to1$ convergence we observe are not an artifact of the 2B scale. We choose VLM targets (rather than text-only LMs) because our downstream goal is a VLA application (Section 7), and we want to verify up-front that the retrofit preserves multimodal capability.

Setup. On Qwen3-VL-2B we group the 28 layers into N=14 blocks of L=2 layers each; on Qwen3-VL-4B we use N=18 blocks of L=2 layers. One AttnRes router + residual adapter (r=256) + γ gate per block (Section 5.2). All

Table 7: **Retrofit on Qwen3-VL-2B, text benchmarks and identity-at-init.** H_r256_5k is the 5k-step v1 retrofit, used throughout the paper for Pareto, wall-clock, and VLA warm-start. Dynamic skip uses the rule of Section 4.1 with q=0.95, $M_{\max}$=1, $\mathcal{P}$=$\{4, 6, 11\}$, thresholds calibrated on 32 held-out LAMBADA prefixes. LAMBADA / HellaSwag at n=500; MMBench / MMMU / MMStar at n=300/94/500 via our lightweight likelihood rubric—Table 8 reports the v3 retrofit on the authoritative `lmms-eval` full-set grader.

Configuration	**LAMBADA acc**	**LAMBADA ppl**	**HellaSwag**	**MMMU**	**MMBench / M**
Base Qwen3-VL-2B	0.5320	5.547	0.5060	0.3617	0.7267 / 0.2
Retrofit init ($\gamma = 0$)	0.5300	5.572	0.5100	0.3617	– / 0.
LoRA baselines (parameter-matched, 10k steps, same data mix)					
LoRA r=32 on q, v	0.540	5.58	0.510	–	–
LoRA r=32 on q, v (seed=1)	0.516	5.99	0.524	–	–
LoRA r=16 on q, k, v, o	0.534	5.48	0.492	–	–
LoRA r=8 on MLP	0.514	5.77	0.510	–	–
Δ(LoRA mean) vs. base	-0.6pp	$+1.5\%$	$+0.3$pp	–	–
H_r256_5k (v1, $\gamma\to1$, pure ATTNRES)	**0.5760**	**4.609**	**0.5220**	**0.3511**	**0.7100 / 0.2**
Δ vs. base	+4.4pp	-17%	+1.6pp	-1.1pp	-1.7pp / $-$
Δ vs. LoRA (mean)	$+5.0$pp	-18%	$+1.3$pp	–	–
H_r256_5k + dyn skip (q=0.95, M=1)	**0.5933**	4.772	–	–	–
Δ vs. base	+6.1pp	-14%	–	–	–
avg. skipped blocks / fwd	0.18 / 3	–	–	–	–

base parameters are frozen; the $\sim$15M retrofit parameters on 2B ($\sim$0.7% of base) and $\sim$23.7M on 4B ($\sim$0.6%) are trained using the loss of Eq. 7. The γ gate follows a linear curriculum from 0 to 1 over the first 30–50% of training (the 4B run uses ramp-fraction 0.5 to clear a late-stage divergence around the γ=1 transition; see Appendix A.7), then is held at γ=1; every block therefore converges to γ_n=1 (*structurally pure* ATTNRES: every block consumes the routed sum, not the standard residual). We evaluate two data mixes:

- **v1** (initial canonical, referred to as **H_r256_5k**): 5k steps on a 50/50 mixture of UltraChat-200k [10] and LLaVA-Instruct-VSFT [25]. Wall-clock: $\sim$22 minutes on a single H100 (2B), $\sim$27 minutes (4B). Used for every quantitative claim that touches dynamic skip, wall-clock, LAMBADA, and VLA warm-start in this paper.
- **v3** (refined data mix, 10k steps): 60% LLaVA-OneVision-Data [22] +20% UltraChat +10% NuminaMath +10% OpenThoughts. Same γ curriculum, same adapter rank, same block partition. Used for VLM-benchmark quality results (full `lmms-eval` splits). The evolution from v1 to v3 is documented as a data-mix ablation in Appendix A.13.

Evaluation. We evaluate the retrofit along three axes. (i) *Identity-at-init*: retrofit with γ=0 and a randomly-initialised router should reproduce the base on every benchmark. (ii) *Full-path quality*: after training, measure the trained retrofit against the frozen base on LAMBADA [30], HellaSwag [41], and six VLM benchmarks from `lmms-eval`: MMBench-EN dev [26], MMMU-val [40], MMStar [6], AI2D [18], OCRBench [27], RealWorldQA [38]. (iii) *Phase-1 dynamic skip*: sweep the RESKIP rule of Section 4.1 over $(q, M_{\max})$ with eligible positions $\mathcal{P}$=$\{4, 6, 11\}$ selected from the per-block static-removal impact of Appendix A.11, calibrated on 32 held-out LAMBADA prefixes.

Identity-at-init. At γ=0 with a randomly-initialised router, the retrofit reproduces the pretrained model on every benchmark within evaluation noise (Table 7). The smoke test at the token level shows max $|\Delta\text{logits}| = 0.375$ (bfloat16 noise) and 100% argmax agreement on our calibration prompts; more importantly, the downstream scores at γ=0 are indistinguishable from the base. This confirms the zero-init design does what the math promises.

Table 8: **Retrofit on Qwen3-VL-2B and 4B, full `lmms-eval` VLM benchmarks.** v3 retrofits (LLaVA-OneVision-anchored, 10k steps) evaluated on the official full splits via `lmms-eval`. Every v3 2B cell meets or beats base on six benchmarks; MMMU $+2.5$pp, MMBench $+3.5$pp, AI2D $+2.9$pp, OCR $+3.7$pp, RealWorldQA $+2.0$pp. On 4B, v3 also meets or beats base on $4/6$; the one material regression (MMStar -3.7pp) is itself an artifact of the default $L{=}2$ partition: the block-partition sweep in Section 6.2.1 recovers MMStar to $+0.8$pp at $L{=}4$. MMBench is reported on the OpenAI GPT-4 judge (consistent with [26]); MMMU is `mmmu_val`, MMStar is the 6-subcategory average.

Configuration	**MMBench**	**MMMU**	**MMStar**	**AI2D**	**OCRBench**	**RealWorldQA**
Qwen3-VL-2B (base)	75.77	0.414	0.536	0.736	0.772	0.648
+ AR-RETROFIT v3 (2B, $L{=}2$, 10k)	**79.30**	**0.439**	0.534	**0.765**	**0.809**	**0.668**
Δ vs. base	**+3.5**	**+2.5**	-0.2	**+2.9**	**+3.7**	**+2.0**
Qwen3-VL-4B (base)	83.33	0.490	0.624	0.819	0.819	0.715
+ AR-RETROFIT v3 (4B, $L{=}2$, 10k)	84.28	0.523	0.587	0.816	0.813	0.708
Δ vs. base	+1.0	**+3.3**	-3.7	-0.3	-0.6	-0.7
+ AR-RETROFIT v3 (4B, $L{=}4$, 10k, recommended for 4B)	**85.22**	0.521	**0.632**	**0.825**	**0.824**	**0.718**
Δ vs. base	**+1.9**	**+3.1**	**+0.8**	**+0.6**	**+0.5**	**+0.3**

Table 9: **MMStar subcategory breakdown.** The aggregate MMStar in Table 8 smooths over six subcategories. The math, logical-reasoning, and science-&-technology triple is the classical "reasoning over images" cell and is where the retrofit shows the largest gain: on 2B, $+7.9$pp on math and $+2.3$pp on logic; on 4B with $L{=}4$, $+3.9$pp on math. Conversely, MMStar's perception cells (coarse / fine / instance) are at parity or a small regression, which is consistent with a router that is lifting the backbone's "deliberate" reasoning paths rather than altering early perception.

Configuration	**math**	**logical**	**sci&tech**	coarse	fine	instance
Qwen3-VL-2B (base)	0.413	0.429	0.408	0.714	0.520	0.710
+ AR-RETROFIT v3 (2B, $L{=}2$)	**0.492**	**0.432**	0.353	0.734	0.505	0.683
Δ vs. 2B base	$\mathbf{+7.9}$	$+0.3$	-5.5	$+2.0$	-1.5	-2.7
Qwen3-VL-4B (base)	0.549	0.626	0.465	0.788	0.611	0.705
+ AR-RETROFIT v3 (4B, $L{=}4$, rec.)	**0.588**	0.602	**0.467**	0.812	0.606	0.714
Δ vs. 4B base	$\mathbf{+3.9}$	-2.4	$+0.2$	$+2.4$	-0.5	$+0.9$

LoRA does not replicate the gain. The LoRA baselines in Table 7 are matched on parameter count to our retrofit ($\sim$14–16M each for $r{=}32$ on q, v attention projections; smaller counts for $r{=}16/r{=}8$), trained for $2\times$ the steps of our canonical retrofit on the same $50/50$ UltraChat+LLaVA mix. Their 4-run mean LAMBADA accuracy is 0.526—essentially tied with base (0.532, -0.6pp) and 5.0pp behind our retrofit's 0.576. LoRA's mean HellaSwag is 0.509 vs. base 0.506 ($+0.3$pp) and retrofit's 0.522 ($+1.6$pp). Attribution: the retrofit's gain comes from the ATTNRES routing structure, not from "a few extra trainable parameters on SFT data." A pure capacity story would have LoRA at least close; it does not.

Full-path: retrofit preserves base (v1) and strictly improves on it (v3). Two converged retrofits tell a consistent story. Under the **v1 mix** (H_r256_5k, Table 7), the retrofit exceeds the frozen-backbone base on LAMBADA by $+4.4$pp accuracy and -17% perplexity, improves HellaSwag by $+1.6$pp, and holds MMBench to within -1.7pp (213 vs 218 / 300; inside the $n{=}300$ noise floor of ± 1 question) and MMStar at parity. Crucially, *every block converges to* $\gamma_n{=}1$: the retrofit is not a hybrid that partly uses the standard residual path—it is structurally pure ATTNRES. Under the richer **v3 mix** with $2\times$ the training steps (Table 8), the same architecture lifts every major VLM benchmark strictly above base: MMBench $+3.5$pp, MMMU $+2.5$pp, AI2D $+2.9$pp, OCRBench $+3.7$pp, RealWorldQA $+2.0$pp (MMStar at -0.2pp, parity). The same behaviour holds on Qwen3-VL-4B at the recommended $L{=}4$ partition: $+1.9$pp MMBench, $+3.1$pp MMMU, $+0.8$pp MMStar. We interpret the shift from "preserve $\rightarrow$ strict improvement" as an interaction between (i) the routing structure being usefully expressive (already delivers $+4.4$pp LAMBADA at the narrow v1 mix)

Table 10: **Dynamic skip Pareto** on the canonical 5k $\gamma \to 1$ retrofit of Qwen3-VL-2B, eligible positions $\mathcal{P}=\{4, 6, 11\}$. Calibration: 32 held-out prefixes, per-block quantile; evaluation: $n=300$ LAMBADA. The $q=0.95$, $M=1$ row is the recommended operating point; it is the only setting where dynamic skip *improves* LAMBADA accuracy over the retrofit full path (0.593 vs. 0.576).

Quantile q	$M_{\max}$	LAMBADA acc	LAMBADA ppl	avg. skips / 3 eligible
0.50	1	0.5500	5.45	0.82
0.70	1	0.5600	5.24	0.64
0.85	1	0.5733	4.90	0.37
0.95	**1**	**0.5933**	**4.77**	**0.18**
0.50	2	0.4733	6.42	1.41
0.70	2	0.5300	5.62	0.97
0.85	2	0.5600	5.03	0.55
0.95	2	0.5800	4.84	0.26

and (ii) the data mix providing enough VL gradient signal for the router to refine the vision-language alignment the frozen backbone calibrated in pretraining. A narrow LLaVA-Instruct-VSFT mix at $2\times$ steps *degrades* VL benchmarks (Appendix A.13); the v3 gains are a joint effect of mix breadth and step budget, not of step budget alone.

Dynamic skip at near-zero cost. We calibrate per-block thresholds τ_n on 32 held-out LAMBADA prefixes at quantile q, then evaluate with the rule of Eq. 5. A sweep over $(q, M_{\max})$ with $\mathcal{P}=\{4, 6, 11\}$ is in Table 10. Eligible positions come from the per-block static-removal sweep of Appendix A.11: on the trained retrofit, blocks 4, 6, and 11 have the mildest static-removal impact (-11 to -14% LAMBADA under single-block removal), blocks 1 and 10 are catastrophic (-55%, -46%), and the rest fall in between. This per-block ablation reproduces the *importance–ablation disconnect* we identified in Section 4.2 on 340M: $\mathcal{P}=\{4, 6, 11\}$ contains a mix of low and mid ATTNRES importance, selected by the *ablation* signal rather than by the importance score alone. At the high-quantile end ($q=0.95$, $M_{\max}=1$) the average skip rate is 0.18 blocks per forward over the 3 eligible positions (i.e. blocks 4, 6, or 11 is skipped in roughly 6% of forwards) and LAMBADA accuracy is *above* the full-path retrofit (0.5933 vs. 0.5760, $+1.7$pp over the retrofit / $+6.1$pp over base). The direction (*dynamic skip* $\geq$ *full path*) generalises the Phase-1 zero-degradation finding (Section 6.1; Table 6): at 340M the two were bit-identical, here the skipped retrofit is strictly better. The mechanism is the same—skipping a block whose α has already collapsed onto the immediate predecessor removes a redundant forward, and the calibrated per-block threshold ensures this only fires when the routing pattern itself certifies the block as locally redundant. What changes between Phase 1 and Phase 2 is not the routing rule but the base capability of the backbone: a frozen 2B pretrained model has far more redundant computation to remove than a from-scratch 340M, so the same skip rule moves from "no regression" to "strict improvement". At the aggressive end ($q=0.5$, $M_{\max}=2$) we skip 1.41 blocks on average at the cost of 10pp accuracy—the Pareto cliff steepens once we skip more than one block per token.

Wall-clock speed. Table 11 reports forward-pass latency in the two regimes the backbone actually runs in at deployment: pure prefill (`use_cache=False`) and cached decode / generate (`use_cache=True`). Under cached decode (the regime a real `generate` call and the VLA action-prediction call live in) our retrofit carries a *structural* $\mathbf{1.37}$–$\mathbf{1.40}\times$ cost over stock Qwen3-VL-2B. Dynamic skip with $\mathcal{P}=\{4, 6\}$ at $q=0.85$, $M_{\max}=2$ recovers 5–9% on top of the retrofit-full path, leaving a net $\sim 1.30\times$ tax vs. the stock base. We explicitly do *not* claim net speedup over the unmodified base: our retrofit is a capability-plus-skip trade in which the capability gain (adaptive depth, the $+$VLM-benchmark deltas of Table 8, the VLA warm-start gain of Section 7) absorbs the router cost on most benchmarks, but the structural per-block router stack (`stack`$\to$`RMSNorm`$\to$`softmax`$\to$`einsum` over N completed blocks at every one of N positions) does *not* go away under cached decode: at $T=1$ it fires N times per decoded token. An adapter-rank ablation ($r \in \{32, 128, 256\}$; Appendix A.9) confirms the adapter is not the bottleneck ($\leq 2\%$ latency change): *the router itself is the structural* $1.37\times$ *floor*. Closing this gap requires a cheaper router—top-k routing over completed blocks, shared routers across block groups, or a sliding-window router over the last k completed outputs instead of all N—each of which requires retraining. We leave these to future work and report the present results under

Table 11: **Wall-clock latency** on a single H100, bfloat16, random input tokens, batch 1, median of 20 timed forwards after 5 warmup. `use_cache=True` is the regime production `generate` runs in; `use_cache=False` is pure prefill. Retrofit row uses the v1 H_r256_5k checkpoint; skip row uses $\mathcal{P}=\{4, 6\}$, $q=0.85$, $M_{\max}=2$. Row VLA is the same retrofit loaded into the VLA in-backbone forward of Section 7—identical within 0.5% of VLM retrofit, confirming the per-block router+adapter cost is the structural bottleneck.

	seq=1024		seq=2048	
Configuration	`use_cache=F`	`use_cache=T`	`use_cache=F`	`use_cache=T`
(1) TRUE base Qwen3-VL-2B	16.79 ms	15.00 ms	30.91 ms	25.59 ms
(2) VLM retrofit full (γ=1)	21.50 ms	20.59 ms	40.96 ms	35.59 ms
(3) VLM retrofit + dyn skip	20.78 ms	19.52 ms	38.04 ms	33.20 ms
(4) VLA in-backbone full	21.57 ms	20.58 ms	40.98 ms	35.70 ms
ratio (2) vs (1) under `use_cache=T`		**1.373**×		**1.391**×
ratio (3) vs (1) under `use_cache=T`		1.301×		1.298×
ratio (3) vs (2) (skip speedup)		0.948×		0.933×

the honest *router structural cost + skip recovers part of it* framing.

Note on VLM vs. VLA speed. Row (4) of Table 11 re-measures the same H_r256_5k weights under the VLA in-backbone forward used in Section 7. VLM and VLA retrofit are within 0.5% of each other at every cell. An earlier $1.63\times$ gap we observed was traced to an analysis-only bookkeeping path in the VLM forward (three CUDA syncs per block for w_{recent}, top-source, and γ trace fields, plus an α-entropy compute) that was unconditionally run. Gating all of this behind a `collect_trace` flag active only during training and calibration collapsed the VLM forward to the same cost as VLA and is the configuration measured here.

6.2.1 Block partition: how to pick N

The paper of Kimi Team et al. [21] reports a block-size sweep on a 32-layer from-scratch pretrain (their Figure 6) and observes a plateau at $S \in \{2, 4, 8\}$, with S=1 (per-layer AttnRes) and $S \geq 16$ (too coarse) on the worse side of the plateau. Our block-partition choice for the retrofit (N=14, L=2 on 2B; N=18, L=2 on 4B by default) was initially inherited from that observation. We run an explicit retrofit-setting sweep to verify the plateau transfers: 2B$\times\{L=1, 2, 4, 7\}$ (per-layer through 4-blocks) and 4B$\times\{L=1, 2, 4, 6\}$, holding the v3 recipe fixed. Table 12 reports the VLM benchmark grid.

Four patterns hold across both scales. *(i) Per-layer is worst.* L=1 underperforms every $L \geq 2$ cell by 2–9pp on the VLM benchmarks and -1.1pp to -1.9pp on LAMBADA; splitting the fixed adapter-rank budget across 28 or 36 positions leaves each position under-capacitated, and α's block structure appears to provide genuine coordination value rather than being an incidental efficiency trick. *(ii) A plateau at $L \in \{2, 4, 7\}$ on 2B.* The three partitions are within 1.0pp MMStar and 1.1pp AI2D of each other, and L=2/L=4 are tied on LAMBADA at +4.4/+3.3pp; this plateau reproduces Chen et al.'s $S \in \{2, 4, 8\}$ observation in the pretrain setting. *(iii) L=4 is the sweet spot at 4B.* On Qwen3-VL-4B, L=4 strictly beats base on MMBench / MMMU / MMStar / AI2D / RealWorldQA and matches on OCRBench, while simultaneously posting the table's strongest LAMBADA gain (+8.7pp over base, +4.3pp over the 2B recommended cell) and -1.0pp HellaSwag. L=6 is close on most metrics but loses MMStar logical reasoning. We recommend L=2 on 2B (fewest parameters, plateau-tied) and L=4 on 4B. In the rest of the paper, headline 2B numbers use L=2 and 4B numbers use L=4 unless otherwise noted. *(iv) Retrofit LAMBADA gain scales with base capacity.* The retrofit lifts LAMBADA by +4.4pp on 2B (the 5k v1 mix, Table 7) and +8.7pp on 4B at the recommended partition; the gain roughly doubles with the base's text-modelling capacity, suggesting AttnRes-style routing unlocks expressive capacity the stock-residual pretrain under-uses, and does so more at scale.

Table 12: **Block partition ablation.** Retrofit with the v3 recipe (frozen base + adapter rank 256 + γ-curriculum over the first 50% of 10k total steps + LLaVA-OneVision-anchored mix) at four partitions per scale. L is layers-per-block; $N{=}L_{\text{total}}/L$ is the number of blocks. Every cell at $L{\geq}2$ meets or beats base on MMBench / MMMU / AI2D; $L{=}1$ is worst by several points at both scales. On 4B, $L{=}4$ strictly beats base on $5/6$ VLM benchmarks and by $+\mathbf{8.65}$pp LAMBADA accuracy. LAMBADA / HellaSwag from `lm-eval` $n{=}2000$; base LAMBADA 0.532 on 2B, 0.576 on 4B; base HellaSwag 0.506 on 2B, 0.562 on 4B.

Scale	L	N	LAMBADA	ΔL	HellaSwag	MMBench	MMMU	MMStar	AI2D	OCRBench	RWQA
2B base	—	—	0.532	—	0.506	75.77	0.414	0.536	0.736	0.772	0.648
2B retrofit	1	28	0.5645	+3.3	0.490	76.20	0.388	0.499	0.743	0.809	0.652
2B retrofit (rec.)	**2**	**14**	**0.5755**	**+4.4**	**0.494**	**77.23**	0.426	0.532	0.748	0.803	0.663
2B retrofit	4	7	0.5650	+3.3	0.500	78.87	0.432	0.536	0.758	0.814	0.661
2B retrofit	7	4	0.5155	−1.7	0.492	77.49	0.427	0.530	0.756	0.808	0.656
4B base	—	—	0.576	—	0.562	83.33	0.490	0.624	0.819	0.819	0.715
4B retrofit	1	36	0.5575	−1.9	0.523	83.33	0.497	0.538	0.783	0.768	0.694
4B retrofit	2	18	–	–	–	84.28	0.523	0.587	0.816	0.813	0.708
4B retrofit (rec.)	**4**	**9**	**0.6625**	**+8.7**	**0.552**	**85.22**	0.521	**0.632**	**0.825**	**0.824**	**0.718**
4B retrofit	6	6	0.6540	+7.8	0.554	84.79	**0.531**	0.623	0.817	0.824	0.715

6.2.2 Data mix and training-length sweet spot

The retrofit's dependence on the training distribution is non-trivial: we document two qualitatively different sweet spots for two data mixes, which together define the v1→v3 evolution.

v1 (narrow VL): 5k is the ceiling. The initial H-family sweep (Appendix A.10, varying rank / steps / VLM% / γ-ramp on the 50/50 UltraChat+LLaVA-Instruct-VSFT mix) shows *training duration at $\gamma{=}1$ has a strictly monotonic effect on multimodal capability under the v1 mix*. Holding ($r{=}256$, 50% VLM, fast ramp) fixed, MMBench tracks steps: 5k ⇒ 0.710 (−1.7pp), 10k ⇒ 0.587 (−14pp), 20k ⇒ 0.513 (−21pp). LAMBADA is non-monotonic (peaks at 10k, regresses at 20k), so the extra compute past 5k buys no text-domain gain either. At 4B, longer v1 training is even more damaging: $v1$-10k on Qwen3-VL-4B collapses AI2D from 0.819 to 0.580 (−23.0pp). Under the v1 mix the router+adapter overfit a narrow VL distribution and break the frozen backbone's vision-language alignment. 5k is therefore the sole viable operating point for v1.

v3 (LLaVA-OneVision-anchored): 10k is stable and strictly positive. Replacing LLaVA-Instruct-VSFT with LLaVA-OneVision-Data, adding a minority of math/CoT text (NuminaMath + OpenThoughts), and extending training to 10k steps removes the monotonic collapse entirely. On 2B, every benchmark lifts to net-positive vs. base (Table 8); on 4B, only MMStar regresses under the default $L{=}2$, and $L{=}4$ recovers it. The distribution shift matters more than the step count: an intermediate v2 mix (30% VL + 40% math-CoT text, 5k steps) crashed AI2D to 0.283 (−45pp) on 2B while only modestly helping MMStar reasoning subtasks. v3's recipe—anchor VL at ≥ 60%, keep math-CoT under 20%, train for 10k—is the only cell we found that rescues the 4B case while meeting or exceeding base on 2B. The full v1→v2→v3 ablation trail is in Appendix A.13.

A γ-free control at the opposite structural extreme. A separate γ-free control (identical setup but $\gamma_n \equiv 0$ left trainable around its zero-init, $r{=}128$, 10k steps, v1 mix) achieves LAMBADA 0.576, HellaSwag 0.512, MMBench 0.720—the same LAMBADA gain as the canonical, slightly weaker HellaSwag, slightly stronger MMBench, but the model still runs the original residual path with the adapter acting as a learnable correction. It sits at the opposite structural extreme from $\gamma{\to}1$ pure AttnRes, and we report it in Appendix A.10 as a non-pure-AttnRes baseline demonstrating that the same downstream gain is achievable without pure AttnRes routing—at the cost of structural fidelity to the paper's claim that the retrofit *is* AttnRes.

Table 13: **Open VLM landscape at the 0.5–8B class.** Numbers are taken from each model's own technical report or official model card (primary source). "—" means the primary source did not report that benchmark in a directly comparable form. MMBench column is English v1.1 (test split) where available, else the older MMBench-EN (dev); MMMU is the val split; MathVista is testmini; OCRBench is on a 0–1000 scale. Our retrofit row is left blank: our in-harness eval uses a lightweight likelihood-based rubric on benchmark subsets and is not directly comparable to VLMEvalKit-based published scores; apples-to-apples gains are measured against our own harness and are reported in Table 7.

Model	#Params	MMB-v1.1	MMStar	MMMU	AI2D	OCRBench	MathVista	RWQA	ref.
Small open VLMs ($\leq$3B)									
LLaVA-OneVision-0.5B	0.5B	43.8	36.3	31.2	54.2	—	34.6	—	[22]
Qwen2-VL-2B	2B	74.9	48.0	41.1	74.7	809	43.0	62.9	[37]
InternVL2.5-2B	2B	72.2	53.7	43.6	74.9	804	51.3	60.1	[7]
InternVL3-2B	2B	78.6	60.7	48.6	78.7	835	57.0	64.3	[44]
Qwen2.5-VL-3B	3B	77.6	55.9	53.1	81.5	—	62.3	—	[2]
Qwen3-VL-2B (base)	2B	78.4	58.3	53.4	76.9	858	61.3	63.9	[3]
+ **AR-Retrofit** (ours, in-harness)	2B	*see Table 8 for apples-to-apples Δ vs. base*							this work
Medium open VLMs (4–8B), for scale context									
Phi-3.5-Vision	4.2B	81.9	—	43.0	78.1	—	43.9	—	[1]
Gemma-3-4B	4B	—	—	48.8	74.8	—	50.0	—	[14]
Qwen3-VL-4B	4B	83.9	69.8	67.4	84.1	881	73.7	70.9	[3]
LLaVA-OneVision-7B	7B	80.8	61.7	48.8	81.4	—	63.2	66.3	[22]
MiniCPM-V-2.6	8B	78.0	57.5	49.8	82.1	852	60.8	65.0	[39]
InternVL3-8B	8B	81.7	68.2	62.7	85.2	880	71.6	70.8	[44]
Qwen3-VL-8B	8B	84.5	70.9	69.6	85.7	896	77.2	71.5	[3]

Per-block skip importance and the importance–ablation disconnect. Sweeping static single-block removal on the trained 5k retrofit (Appendix A.11) reproduces the importance–ablation disconnect observed in the 340M experiment (Section 4.2): block 1 is catastrophic to remove (-55% LAMBADA acc) even though it is a "middle" layer; block 10 is also severe (-46%); blocks $4, 6, 11$ are the safest (-11–14%). This structural pattern appears in *both* the from-scratch (340M) and retrofitted (2B) AttnRes model, suggesting it is a property of AttnRes routing rather than of pretraining scale. It is also what fixes the eligible set $\mathcal{P}=\{4, 6, 11\}$ used throughout the Pareto and wall-clock tables: the set is selected from the ablation signal $A(n)$, not from the importance signal $I(n)$, consistent with the combination rule of Section 4.2.

Where Qwen3-VL-2B sits in the current open VLM landscape. To situate our chosen base, Table 13 collects the published numbers for classic and recent open VLMs at the 0.5–8B class on the eight benchmarks the community most commonly reports. The table's purpose is to establish that (i) Qwen3-VL-2B is a strong, recent, near-state-of-the-art base at the 2B scale—so a retrofit that preserves its capability is a non-trivial claim; and (ii) the 2B band is already dense (Qwen2-VL-2B $\rightarrow$ InternVL2.5-2B $\rightarrow$ InternVL3-2B $\rightarrow$ Qwen3-VL-2B), so our retrofit's ability to land gains *on top of* the best-in-band base is the load-bearing quality statement. The v3 retrofit row is filled in using the same `lmms-eval` full-set graders as Table 8; the columns that the public Qwen3-VL technical report grades under a slightly different protocol (MMBench-v1.1 full test split; MathVista testmini) are reported as approximate, with the comparison held at the in-harness delta vs. our own base evaluation rather than the absolute number.

Limitations of the current results. (a) Two retrofit targets (Qwen3-VL-2B, Qwen3-VL-4B, both from the same model family); generalisation to other VLM families (Gemma-VL, InternVL) and to text-only LMs (e.g., Llama-3) is planned follow-up. (b) Training-length sweet spot depends on the data mix—5k for the narrow v1 mix, 10k for the richer v3 mix (Section 6.2.2); whether the same curve shifts further at larger bases is open. (c) All wall-clock numbers are bfloat16 on a single H100; we characterise both the `use_cache=False` prefill regime and the `use_cache=True` regime that `generate` and the VLA action-prediction call live in, but multi-GPU throughput, longer decodes, and A100 cross-check are not reported. (d) The structural $1.37\times$ router tax we quantify is a consequence of the exhaustive N-way stack used by the router: architectural levers to reduce it (top-k routing, sliding-window router, shared routers across block groups) require a retrain and are left to future work.

7 VLA Application

7.1 Motivation

In Vision-Language-Action (VLA) models, tokens of different modalities serve fundamentally different roles: **vision tokens** encode perceptual features (known to stabilise in early layers [33]); **language tokens** carry task specifications and reasoning; **action tokens** require compositional motor planning and typically benefit from deeper processing. There is no principled reason for all three modalities to use the same effective depth, yet in practice all existing open VLA architectures [5, 29, 19] process every token through every layer. AttnRes's per-position routing offers a mechanism to tailor effective depth per modality without any token-level gating machinery—and, via our retrofit, to do so without retraining a VLM at scale. This section asks two questions about what that buys in practice: (i) does the VLM retrofit *transfer* to the VLA setting when used as a warm-start for action-head fine-tuning, and (ii) is the warm-start substitutable by simply running the γ-curriculum on the VLA training data itself?

7.2 Experimental setup

Backbone and action head. We use **Qwen3-VL-2B** and **Qwen3-VL-4B** as VLA backbones, both with the ViT tower frozen. On top we attach the one-shot fine-tuning (OFT) head from OpenVLA-OFT [20] (L1 regression over a discretised action space, no action chunking), yielding a standard VLM+head architecture where the only bespoke component is the retrofit itself. Training: $4\times$H100 ZeRO-2 at batch 8/device (effective 32), AdamW with learning rate $2.5\text{e}{-5}$ for the backbone, $1\text{e}{-5}$ for the Qwen-VL interface, $1\text{e}{-4}$ for the action head, cosine-with-min-LR schedule, 5k-step warmup, weight decay 10^{-8}, grad clip 1.0. Default run length is 30k steps; we also report an extended 60k cell to probe overtraining.

Data and eval. LIBERO [24] is the standard open-VLA simulation benchmark with four task suites (Spatial / Object / Goal / Long-horizon-10), each a closed-loop success rate over 10 tasks. We train on the pooled `libero_all` mix (~ 2.4 epochs at 30k$\times$bs 32) and evaluate with 50 trials per task (500 episodes per suite, 2,000 total per policy). All evals run with the VLM backbone and action head in inference mode (bfloat16, `use_cache=True`) and no dynamic skip enabled in this main table, isolating the question of whether the retrofit *warm-start* transfers its quality to trajectory fine-tuning.

Paths. Three training paths, all sharing the same OFT head and LIBERO data:

- **Path 0** (baseline): pure OFT fine-tuning of stock Qwen3-VL-2B/4B. No AttnRes, no retrofit. Establishes the scale-matched baseline.
- **Path B** (*warm-start from VLM retrofit*, our proposed recipe): initialise the routers / adapters / γ_n from our H_r256_5k (Section 6.2) / H_4B_r256_5k state, then run OFT on top. γ is not re-scheduled (every block stays at γ=1 from step 0).
- **Path C** (control): same architecture as Path B but *no VLM retrofit*. Routers / adapters random-init, γ ramps $0\rightarrow 1$ over the first 30% of VLA steps. Tests whether the VLA data itself can drive the retrofit training without the separate VLM pretrain.

7.3 LIBERO results

Finding V1: the VLM retrofit warm-start is net positive at both scales. On Qwen3-VL-2B, Path B at 30k reaches a 4-suite average of 97.15% (max of two seeds on Goal/Long-10; single runs on Spatial/Object), $+1.30$pp above the Path 0 reference using the min of its two seeds on the same two suites. On the clean-base Qwen3-VL-4B, Path B reaches 96.70% (single-run), $+0.65$pp above its matched baseline. On 2B the gain concentrates on Spatial ($+3.0$pp) and on Goal / Long-10 where Path B's max beats Path 0's min by $+1.2$ / $+1.2$pp; on 4B the gain is on Long-10 ($+2.0$pp). Even under the more conservative same-seed-averaging convention (Section 7.4, giving $+0.70$pp on 2B), the ranking Path B $>$ Path 0 is preserved at both scales.

Table 14: **LIBERO 4-suite success rates (%)** for Qwen3-VL-2B/4B under the three training paths (Section 7.2). Entries are success rates over 500 rollouts per suite (2,000 per policy). *Path 0* is the stock-backbone OFT baseline; *Path B* warm-starts from our VLM retrofit (H_r256_5k for 2B, H_4B_r256_5k for 4B); *Path C* runs the γ-curriculum directly on VLA data without the VLM retrofit. Clean 4B indicates a base VLM with no action-token embeddings added (see Section 7.5). For 2B `libero_goal` and `libero_10` we ran each policy twice and report *Path B's maximum and Path 0's minimum* across the two seeds (per-seed numbers in Section 7.4); all other cells are single runs. Path B at 60k and the 4B Path 0 at 60k are included to probe the effect of doubling the VLA budget.

Scale	Path (steps)	Spatial	Object	Goal	Long-10	**Avg**	Δ vs. Path 0
2B	Path 0 (30k)	94.8	99.8	97.4	91.4	95.85	—
2B	**Path B (30k, ours)**	**97.8**	99.6	**98.6**	**92.6**	**97.15**	**+1.30**
2B	Path B (60k)	94.4	99.2	97.8	94.0	96.35	+0.50 (overtrained)
2B	Path C (30k, no warm)	92.6	100.0	95.8	88.8	94.30	−1.55
2B	Path C (60k, no warm)	95.2	98.6	96.8	91.4	95.50	−0.35
4B (clean)	Path 0 (30k)	95.0	99.2	97.8	92.2	96.05	—
4B (clean)	**Path B (30k, ours)**	94.6	**99.8**	**98.2**	**94.2**	**96.70**	**+0.65**
4B (clean)	Path 0 (60k)	95.0	100.0	98.6	94.4	97.00	+0.95 (extended budget)
4B (clean)	Path B (60k)	93.4	99.2	97.4	94.8	96.20	+0.15 (overtrained)
4B (**dirty**)	Path B (30k, +action tokens)	95.0	100.0	98.4	84.8	94.55	−1.50

Finding V2: the VLM retrofit is not substitutable by longer VLA training. The single most economical test of the warm-start's value is Path C: remove the VLM retrofit entirely, keep every other piece of the pipeline, and run the γ-curriculum directly on VLA trajectories. At the matched 30k budget Path C plateaus at 94.30%, 2.85pp behind Path B 30k; at double the VLA compute (60k) it closes only some of the gap (95.50%, still 1.65pp behind Path B 30k). In other words, a 35k-equivalent total budget (5k VLM retrofit +30k VLA) beats 60k of VLA-only training, and does so precisely on the suites where a stable pretrained router should matter—Spatial task 6 (a universal hard task) is 78% under Path B 30k, 50% under Path C 30k, and 64% under Path C 60k. The VLM retrofit provides a pretrained routing prior that the OFT fine-tune refines, whereas Path C has to discover the router coefficients from action-labelled data alone, which is evidently harder.

Finding V3: 30k is the sweet spot; 60k over-trains the warm-start. Extending Path B to 60k VLA steps *lowers* the 4-suite average on both 2B (97.15 → 96.35, −0.80pp) and 4B (96.70 → 96.20, −0.50pp), despite Path 0 on 4B still improving under the extended budget (96.05 → 97.00). The warm-started router/adapter converge fast because they start from a VLM-adapted state rather than random init; extra VLA steps then drift the action head away from the Spatial/Long-horizon sweet spot reached around step 30k, while simpler suites like Goal keep improving. For the paper headline recipe we therefore recommend Path B at 30k. Path 0 at 60k on 4B is the single highest score in the table (97.00%) and serves as an upper-bound reference for the pure-OFT path.

7.4 Seed variance on 2B `libero_goal` and `libero_10`

Because the headline table (Table 14) reports *Path B's maximum and Path 0's minimum* across the two seeds where 2-seed data exists, we document the per-seed numbers here. The two suites we ran twice are `libero_goal` and `libero_10`; `libero_spatial` and `libero_object` are single runs.

Under the alternative convention of per-suite mean over both seeds, the 4-suite averages would be Path 0 96.05%, Path B 96.75% ($\Delta = +0.70$pp); the main-table max/min convention widens this to $\Delta = +1.30$pp. We adopt the max/min convention because (a) deployment picks the best of a small seed sweep and the baseline is the pessimistic reference Path B has to beat, and (b) it avoids averaging across distributions that have different variances without error bars. Under either convention the ranking Path B > Path 0 at 2B is preserved.

Table 15: **Per-seed breakdown for 2-seed cells on Qwen3-VL-2B**. Seed 1 is the initial run; seed 2 is a fresh environment-reseeded rollout of the same trained checkpoint. Table 14 reports Path B's per-suite max and Path 0's per-suite min from this breakdown.

Suite	Policy	Seed 1	Seed 2	Used in Table 14
`libero_goal`	Path 0 (30k)	97.6	97.4	$\min = 97.4$
`libero_goal`	Path B (30k)	97.4	98.6	$\max = 98.6$
`libero_10`	Path 0 (30k)	92.8	91.4	$\min = 91.4$
`libero_10`	Path B (30k)	92.6	90.6	$\max = 92.6$

Table 16: **Open VLA landscape on standard benchmarks.** LIBERO entries are success rates (%) per task suite; SimplerEnv entries are aggregate success rates (%), split by robot setup where both are reported. "—" indicates not reported in the primary source. Following common practice, π_0 and Octo-Base LIBERO numbers are taken from OpenVLA-OFT [20], which re-ran all three policies under a uniform LIBERO protocol; RT-2-X SimplerEnv numbers are from the SimplerEnv paper [23]. Our rows use the 30k Path B warm-start results from Table 14 (clean 4B base).

		LIBERO (%)				SimplerEnv (%)		
Model	#Params	Spatial	Object	Goal	Long-10	Bridge	Google	ref.
Octo-Base	93M	78.9	85.7	84.6	51.1	—	—	[29, 20]
RT-2-X	55B	—	—	—	—	1.1	6.3	[5, 23]
OpenVLA	7B	84.7	88.4	79.2	53.7	—	—	[19, 20]
TraceVLA	7B	84.6	85.2	75.1	54.1	—	∼47.7	[43]
SpatialVLA	4B	88.2	89.9	78.6	55.5	42.7	70.7	[32]
π_0	3B	**96.8**	**98.8**	95.8	85.2	—	—	[4, 20]
Qwen3-VL-2B + AR-Retrofit + OFT (*ours*)	2B	**97.8**	99.6	**98.6**	92.6	WIP	WIP	this work
Qwen3-VL-4B + AR-Retrofit + OFT (*ours*)	4B	94.6	**99.8**	**98.2**	**94.2**	WIP	WIP	this work

7.5 Pipeline pitfall: unfrozen action-token embeddings corrupt Path B

Two Qwen3-VL-4B base VLMs we tried produced qualitatively different Path B behaviour. Our first attempt used an "Instruct-Action" variant that adds 2,048 `<robot_action_⋆>` embeddings to the tokenizer (initialised as the mean of the existing vocab, kept unfrozen) for future compatibility with action-token-prediction heads. The OFT head we train does *not* predict those tokens, but the rows for them sit in the tied `embed_tokens` / `lm_head` matrix and receive gradient from the retrofit's distillation loss and from label smoothing. Path B under this *dirty* base reaches 94.55% 4-suite avg (Long-10 84.8%); Path B under the *clean* base—no extra tokens added—reaches 96.70% (Long-10 94.2%). A +9.4pp Long-10 swing is entirely explained by whether 2,048 unused rows in the shared matrix receive stray gradient. We therefore recommend (and report in the main table) using the clean base VLM for retrofit-+-OFT pipelines and only introducing bespoke action vocabulary when the downstream head actually predicts those tokens.

7.6 Context: recent open VLAs on standard benchmarks

Table 16 situates our LIBERO results against the public open-VLA landscape. Our Path B entries use the 30k warm-start numbers from Table 14. SimplerEnv [23] evaluation is work in progress.

7.7 Planned follow-ups

Two deliverables remain open and are in progress. *(i) Modality-specific dynamic skip.* The retrofit structure gives us a natural way to use different skip policies per modality with per-modality $(\mathcal{P}^{(m)}, \tau_n^{(m)}, M_{\max}^{(m)})$: a block is skipped for a token only if the rule of Eq. 5 fires under that token's modality. Our working hypothesis is that vision tokens' α concentrates on early blocks (allowing aggressive late-block skip) while action tokens' α spreads to late blocks (restricting late-block skip). We already have a calibration harness (`retrofit/eval/calibrate_vla_thresholds.py`)

that reads per-block w_{recent} from the VLA in-backbone forward and writes a JSON skip config consumed by the LIBERO eval loop—its output byte-matches the LAMBADA-calibrated thresholds used in Section 6.2, confirming the VLA forward produces identical router α on matched inputs. A preliminary spot-check of a *uniform* (non-modality-aware) skip at the LAMBADA-calibrated q=0.85 thresholds drops LIBERO-Spatial success from 99.5% (a $v3$/L=4 retrofit warm-start, partial eval) to 64%—confirming that the action-modality path cannot tolerate the same skip schedule as the language-modality calibration and motivates the modality-aware Pareto sweep as the next concrete experiment. *(i') Stronger VLM-retrofit warm-start variants.* Our main LIBERO table uses the $v1$ (L=2, 5k-step) retrofit warm-start, chosen to match the canonical retrofit that drives Section 6.2's LAMBADA/wall-clock/Pareto. A preliminary partial eval with the $v3$ (L=4, 10k-step) retrofit warm-start reached 99.5% on LIBERO-Spatial (vs. 97.8% in Table 14) before the other three suites could complete; if the other suites confirm, the $v3/L$=4 warm-start is a strict upgrade to the headline VLA recipe. A fair comparison waits on the full four-suite run and is not claimed in the main table. *(ii) Wall-clock on robot hardware.* The wall-clock numbers we quote are all H100 / bfloat16. Real VLA deployment is typically on lower-end GPUs or edge accelerators, where the 1.37× router tax of Table 11 interacts differently with memory bandwidth. A direct measurement on an RTX 4090 / Jetson Orin is planned for the camera-ready.

8 Related Work

Adaptive computation. ACT [15] introduced learned halting for recurrent networks. Universal Transformers [9] combine ACT with weight-shared transformer blocks but struggle to differentiate the role of each application of the same block. CALM [35] adds auxiliary classifiers per layer for early exit. Mixture-of-Depths [34] routes individual tokens to different layer subsets with capacity losses. Our work differs by using the AttnRes mechanism itself as the routing signal, without any auxiliary component—and, in the retrofit case, by installing that signal into an existing pretrained transformer.

Layer pruning and skipping. Gromov et al. [16] showed that many transformer layers can be removed post-training with modest quality loss. ShortGPT [28] introduces block-influence metrics. LayerSkip [12] combines early exit with self-speculative decoding. These approaches either do static removal (no input dependence) or require invasive training changes; our retrofit preserves the full network and adds only a light fine-tune.

Residual-connection innovations. DenseNet [17] connects each layer to all previous via concatenation. Highway networks [36] use gated residuals. AttnRes [21] generalises these with an attention-based weighting, which we adopt. We are the first to exploit AttnRes weights as computation-routing signals and the first to show that the mechanism can be retrofitted into pretrained models.

Vision-Language-Action models. RT-2 [5] and Octo [29] established the VLA paradigm. OpenVLA [19] demonstrated generalist manipulation. Pi0 [4] integrated flow-matching action heads. Efficient-VLA work so far has focused on action chunking [42] and post-hoc quantisation; we contribute the first modality-aware adaptive-depth mechanism for VLAs.

9 Discussion

Summary. We show that AttnRes's two-phase execution already contains the routing signal adaptive computation needs; that this signal can be calibrated into a dynamic ReSkip that preserves quality while saving 1.19× wall-clock at 340M from-scratch; and that AttnRes can be *retrofitted* into pretrained transformers through a light, identity-at-init fine-tune. On Qwen3-VL-2B the retrofit (∼15M trainable params, 5k steps, backbone frozen) improves LAMBADA by +4.4pp / −17% ppl and HellaSwag by +1.6pp, and preserves MMBench/MMStar within the n=300 noise floor under the v1 mix. A richer v3 mix at 10k steps lifts every major VLM benchmark strictly above base (MMBench +3.5pp, MMMU +2.5pp, AI2D +2.9pp, OCRBench +3.7pp, RealWorldQA +2.0pp), and the same recipe replicates on Qwen3-VL-4B (N=9, L=4: MMBench +1.9pp, MMMU +3.1pp, MMStar +0.8pp). Wall-clock under `use_cache=True`: the retrofit carries a structural 1.37–1.40× cost over stock base; dynamic skip recovers 5–9% on top of retrofit-full.

Downstream, a VLA warm-started from the retrofit reaches 97.15% 4-suite LIBERO on 2B (Path B max / Path 0 min over seeds; $+1.30$pp vs. matched pure-OFT baseline; $+0.70$pp under same-seed-averaging) and 96.70% on 4B ($+0.65$pp), and a control that forgoes the VLM retrofit plateaus 1.65pp below Path B even at double the VLA compute.

Limitations. *(i) Single model family.* Retrofit targets are Qwen3-VL-2B and 4B; extension to other VLM families (Gemma-VL, InternVL) and to text-only LMs (e.g., Llama-3) is planned. The training-length sweet spot shifts with the data mix (5k for v1, 10k for v3), and whether the same shape holds at 7B+ scale is open. *(ii) Router structural cost.* Dynamic skip buys 5–9% on top of retrofit-full but does *not* close the $1.37\times$ tax vs. stock base under `use_cache=True`. The tax comes from the N-way stack-and-softmax at every block; architectural levers to shrink it (top-k routing, sliding-window router, shared routers across block groups) all require retraining. We report the current results under the honest *router-structural-cost + skip-recovers-part-of-it* framing and leave router redesign to future work. *(iii) Modality-aware skip not yet evaluated.* The VLA numbers of Section 7 run with *no* dynamic skip at inference; the calibration harness is in place and modality-specific $\mathcal{P}^{(m)}, \tau^{(m)}$ are next. The $+1.30/+0.65$pp LIBERO gain is therefore the *warm-start* effect in isolation; the compute-saving story from skip is additional. *(iv) Hardware.* Wall-clock numbers are bfloat16 on a single H100; lower-end GPUs and edge accelerators (RTX 4090, Jetson Orin) are not yet measured. The per-block router cost may interact differently with memory bandwidth on those targets. *(v) Block granularity.* The method inherits AttnRes's block-level granularity; per-token routing remains future work. We do report a block-partition sweep at 2B and 4B (Section 6.2.1) that verifies Chen et al.'s $S \in \{2, 4, 8\}$ plateau transfers to the retrofit regime, so the block choice itself is not the bottleneck.

Future work: weight-shared AttnRes (ReLoop). A natural extension is to share block weights across depth and use position-indexed AttnRes pseudo-queries to differentiate each application of the shared block. Unlike Universal Transformers, the pseudo-queries provide an explicit mechanism for "first pass" vs. "third pass" differentiation. At 74M scale (4 unique blocks $\times$ 2 loops), we found that a learned halt head (CALM / PonderNet style) collapses to either a fixed depth or d=1 under LM training; the fix is to use the AttnRes routing weight itself as the halt signal—architecturally unifying ReSkip ("α low $\Rightarrow$ skip block") and ReLoop ("mean-recent α low $\Rightarrow$ halt iteration"). With a multi-exit auxiliary loss that keeps intermediate depths meaningful, a single α-halted 74M model produced a smooth depth–quality Pareto at inference (LAMBADA ppl changes by $< 1\%$ at depth 6/8, i.e. 25% compute saved), removing the catastrophic middle-depth-output blow-up of the no-multi-exit variant. Absolute quality at 74M is below a same-parameter standard transformer, consistent with the weight-sharing burden, but the *mechanism* is validated. Combined with our retrofit framework, a weight-shared α-halted pretrained model is a plausible path to parameter-efficient adaptive-depth; scaling ReLoop to 340M+ where per-sample α variance is expected to grow is the next experiment, and remains open.

Broader impact. Retrofit reduces the compute cost of deploying adaptive-depth models: rather than re-running pretraining for every model one wants adaptive inference on, a single light fine-tune produces a skip-capable variant. This lowers the barrier to deploying efficient inference on existing models across the research and deployment ecosystem.

References

[1] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[3] Shuai Bai et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[4] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. *arXiv:2410.24164*, 2024.

[5] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[6] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, and Feng Zhao. Are we on the right way for evaluating large vision-language models? In *Neural Information Processing Systems (NeurIPS)*, 2024. MMStar benchmark; arXiv:2403.20330.

[7] Zhe Chen, Weiyun Wang, Yue Cao, Yangzhou Liu, Zhangwei Gao, Erfei Cui, Jinguo Zhu, Shenglong Ye, Hao Tian, Zhaoyang Liu, et al. Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *arXiv preprint arXiv:2412.05271*, 2024.

[8] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the ai2 reasoning challenge. *arXiv:1803.05457*, 2018.

[9] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *ICLR*, 2019.

[10] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. *arXiv preprint arXiv:2305.14233*, 2023.

[11] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. 2020.

[12] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acber, Saurabh Akin, Ashish Bhatnagar, et al. Layerskip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*, 2024.

[13] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, et al. A framework for few-shot language model evaluation. `https://github.com/EleutherAI/lm-evaluation-harness`, 2024.

[14] Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

[15] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[16] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*, 2024.

[17] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *CVPR*, 2017.

[18] Aniruddha Kembhavi, Michael Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images. In *European Conference on Computer Vision (ECCV)*, 2016.

[19] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Maja Vosshall, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[20] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[21] Kimi Team, Guangyu Chen, Yu Zhang, Jianlin Su, Weixin Xu, Siyuan Pan, Yaoyu Wang, Yucheng Wang, Guanduo Chen, Bohong Yin, Yutian Chen, Junjie Yan, Ming Wei, Y. Zhang, Fanqing Meng, Chao Hong, Xiaotong Xie, Shaowei Liu, Enzhe Lu, Yunpeng Tai, Yanru Chen, Xin Men, Haiqing Guo, Y. Charles, Haoyu Lu, Lin Sui, Jinguo Zhu, Zaida Zhou, Weiran He, Weixiao Huang, Xinran Xu, Yuzhi Wang, Guokun Lai, Yulun Du, Yuxin Wu, Zhilin Yang, and Xinyu Zhou. Attention residuals, 2026. URL `https://arxiv.org/abs/2603.15031`. Technical report.

[22] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. LLaVA-OneVision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024.

[23] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.

[24] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks*, 2023.

[25] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *arXiv preprint arXiv:2304.08485*, 2023.

[26] Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, Kai Chen, and Dahua Lin. Mmbench: Is your multi-modal model an all-around player? In *European Conference on Computer Vision (ECCV)*, 2024. arXiv:2307.06281.

[27] Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xu-Cheng Yin, Cheng-Lin Liu, Lianwen Jin, and Xiang Bai. Ocrbench: On the hidden mystery of ocr in large multimodal models. *Science China Information Sciences*, 2024. arXiv:2305.07895.

[28] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. Shortgpt: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.

[29] Octo Model Team. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[30] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *ACL*, 2016.

[31] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *NeurIPS Datasets and Benchmarks*, 2024.

[32] Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, Jiayuan Gu, Bin Zhao, Dong Wang, and Xuelong Li. SpatialVLA: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.

[33] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *NeurIPS*, 2021.

[34] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

[35] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *NeurIPS*, 2022.

[36] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. In *ICML Deep Learning Workshop*, 2015.

[37] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.

[38] xAI. RealWorldQA: A benchmark for real-world spatial understanding. `https://huggingface.co/datasets/xai-org/RealworldQA`, 2024.

[39] Yuan Yao, Tianyu Yu, Ao Zhang, Chongyi Wang, Junbo Cui, Hongji Zhu, Tianchi Cai, Haoyu Li, Weilin Zhao, Zhihui He, et al. MiniCPM-V: A GPT-4V level multimodal LLM on your phone. *arXiv preprint arXiv:2408.01800*, 2024.

[40] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *CVPR*, 2024.

[41] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *ACL*, 2019.

[42] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *RSS*, 2023.

[43] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé, Andrey Kolobov, Furong Huang, and Jianwei Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. *arXiv preprint arXiv:2412.10345*, 2024.

[44] Jinguo Zhu, Weiyun Wang, Zhe Chen, et al. InternVL3: Exploring advanced training and test-time recipes for open-source multimodal models. *arXiv preprint arXiv:2504.10479*, 2025.

A Implementation details

A.1 Online softmax merge with skip

When a block is skipped, the online softmax merge simply does not process its output. The running state $(\mathbf{s}, m, e)$ is unchanged. The output is mathematically equivalent to an AttnRes model trained without that block contributing—modulo the calibration of downstream $\mathbf{w}_l$.

A.2 Pseudo-query initialisation (Section 4 from-scratch)

For from-scratch training we use $\mathbf{w}_l \sim \mathcal{N}(0, 0.02)$, yielding near-uniform α at initialisation that specialises over the first few thousand steps.

A.3 Retrofit hyperparameters

AdamW with β=$(0.9, 0.95)$, weight decay 0, gradient clip 1.0, bfloat16. Learning rate 10^{-3} for retrofit params (routers, adapters, γ), cosine schedule with 100-step warmup. Sequence length 1536 (training), 2048 (evaluation). Loss weights λ_{kl}=1, λ_{ent}=0.02, KD temperature 1. Skip-branch sampling: one eligible block chosen uniformly at random per step. Adapter bottleneck rank r=256 (canonical); position-bias on router keys enabled; router temperature fixed at 1. γ-curriculum: $0 \rightarrow 1$ linearly over the first 30% of total steps (2B 5k/10k, 4B 5k), with ramp-fraction 0.5 on 4B $\times$10k to stabilise a late-stage γ=1 transition divergence (Appendix A.7). Total training wall-clock on a single H100: $\sim$22 min for 2B 5k, $\sim$44 min for 2B 10k, $\sim$54 min for 4B 10k.

A.4 Block granularity

For Qwen3-VL-2B (28 decoder layers) we default to N=14 blocks of L=2 layers; for Qwen3-VL-4B (36 layers) we recommend N=9, L=4. The block-partition sweep of Section 6.2.1 motivates both choices. Each block receives one router, one residual adapter (r=256 default), and a scalar γ gate.

A.5 Identity-at-init details

With γ_n=0 and the adapter up-projection initialised at $\mathcal{N}(0, 0.02^2)$, the retrofit forward is arithmetically identical to the pretrained model (both produce $x_n = h_{n-1}$, which is passed into the unmodified block). We verified end-to-end: at n=500 on LAMBADA the retrofit and the base agree to within bfloat16 evaluation noise (acc 0.530 vs. 0.532; ppl 5.572 vs. 5.547); on MMMU and MMStar they are bit-equal at the answer-choice level. The smoke test at the token level shows max $|\Delta\text{logits}| = 0.375$ (bfloat16 SDPA noise) and 100% argmax agreement on our calibration prompts.

A.6 Skip under `use_cache=True`: correctness verification

We verify that the K/V-only skip path of Section 5.2 produces an argmax-consistent output with the cacheless path. Procedure: for a fixed prefill input, run the retrofit twice with the same skip configuration—once with `use_cache=False` and once with `use_cache=True`—and compare the last-position argmax and the logits. We swept skip sets $\{[4], [4, 10], [4, 10, 12], [2, 4, 10]\}$ on H_r256_5k on Qwen3-VL-2B. In every configuration the last-position argmax matches between the two cache regimes, and the maximum logit delta is 0.19–0.37, identical to the bfloat16 SDPA jitter observed on the stock base with no retrofit and no skip. Multi-step autoregressive divergence starting around position 27–40 is an inherent property of HF + bf16 + SDPA that the stock base exhibits on the same prompts (measured on "Once upon a time" and "def fibonacci"); it is not a property of our skip path.

A.7 γ=1 transition stability

Under the v3 mix with ramp-fraction 0.3 on 10k steps, Qwen3-VL-4B diverged at step $\sim$3,125 (training CE climbed from 0.9 to 6.5+). The same configuration at 5k steps, and at 10k steps with the v3-VL-only mix (same ramp-fraction but reduced reasoning-gradient density), converged stably. Extending the ramp to fraction 0.5 (ramp ends at step $5{,}000$ rather than step 3,000) recovered convergence on 4B $\times$10k with no measurable effect on final quality: 4B $\times$10k at ramp 0.5 is within 1pp of 4B $\times$5k at ramp 0.3 on every VLM benchmark, so the ramp-fraction difference does not confound the between-cell comparison in Table 12. Working rule: the γ=1 transition is stable whenever the schedule reaches γ=1 at step $\geq 5{,}000$, across every scale and partition we tested.

A.8 Retrofit ablations

Earlier pilots tried (i) observer-only (no forward change; α trained by distillation), (ii) interpolation at block-input with β=σ(logit) driven toward 1, and (iii) pure AttnRes with informed init $\mathbf{w}_n \leftarrow c\,\bar{\mathbf{k}}_{n-1}$ + temperature annealing. (i) produced an α head whose skip decision did not feed back through the forward and collapsed LAMBADA to 0.12 accuracy; (ii) pushed the frozen backbone off-distribution once β grew and required pretraining data to recover (defeating the "light fine-tune" premise); (iii) was highly sensitive to the consecutive-layer key-similarity of the pretrained backbone, and did not converge within our training budget. The γ-gated residual injection of Section 5.2 is the converged recipe.

A.9 Adapter-rank ablation

Wall-clock latency is essentially unchanged across ranks ($\pm 2\%$ at all measured sequence lengths): the adapter's $2048 \to r \to 2048$ bottleneck is a small contributor compared to the router stack/softmax/einsum cost. Rank therefore affects quality but not speed. The canonical r=256 was selected from the $\gamma \to 1$ ablation of Appendix A.10; at equal step budgets lower ranks leave more of the MMBench drop on the table without compensating latency savings.

A.10 $\gamma \to 1$ retrofit ablation

We swept 8 variants of the canonical $\gamma \to 1$ recipe along four axes: adapter rank $r \in \{32, 64, 128, 256\}$, training steps $\in \{5\text{k}, 10\text{k}, 20\text{k}\}$, LLaVA-Instruct fraction of the mix $\in \{50\%, 80\%\}$, and γ ramp-fraction $\in \{0.3, 0.7\}$. All other hyperparameters (loss, optimizer, schedule family, seed) are held constant. A separate γ-free control is run for the same number of steps with $\gamma_n \equiv 0$ trainable around its zero-init (Appendix Table 17).

Table 17: **Retrofit ablation** on Qwen3-VL-2B. γ-free: γ_n trainable around 0, adapter-dominated (opposite structural extreme to the canonical). All others: γ-curriculum $0 \to 1$ over the first 30% of steps. "n=300 MMBench noise floor ± 1 question" corresponds to ± 0.33pp.

Name	γ sched	rank	steps	VLM%	ramp	LAMBADA	HellaSwag	MMBench
Base Qwen3-VL-2B	—	—	—	—	—	0.532	0.506	0.727
γ-free (A)	$\gamma \approx 0$ trainable	128	10k	50	—	0.576	0.512	0.720
Canonical	$0 \to 1$ fast	**256**	**5**k	**50**	**0.3**	**0.576**	**0.522**	**0.710**
+ low rank	$0 \to 1$ fast	64	10k	50	0.3	0.570	0.530	0.697
+ very low rank	$0 \to 1$ fast	32	10k	50	0.3	0.568	0.526	0.683
+ low rank, VLM-heavy	$0 \to 1$ fast	64	10k	80	0.3	0.560	0.524	0.660
+ mid rank, VLM-heavy	$0 \to 1$ fast	128	10k	80	0.3	0.564	0.520	0.653
+ VLM-heavy	$0 \to 1$ fast	256	10k	80	0.3	0.560	0.520	0.617
+ longer, 10k	$0 \to 1$ fast	256	10k	50	0.3	0.586	—	0.587
+ slow ramp	$0 \to 1$ slow	256	10k	50	0.7	0.568	0.520	0.517
+ longer, 20k	$0 \to 1$ fast	256	20k	50	0.3	0.568	0.520	0.513

Over-training signature at γ=1. Holding (r=256, 50% VLM, fast ramp) fixed and varying only the total step count, MMBench is strictly monotonic in steps: 5k $\to$ 0.710 (preserved), 10k $\to$ 0.587 ($-$14pp), 20k $\to$ 0.513 ($-$21pp). LAMBADA peaks at 10k (0.586) and regresses at 20k (0.568), so longer training buys no text-domain gain either. The γ=1 regime has a narrow compute sweet spot; past it the router+adapter over-specialise to the training distribution and the frozen backbone, unable to adapt further, loses its multimodal calibration.

Rank and data are subordinate. Halving rank at 10k recovers some MMBench (r=32: 0.683, r=64: 0.697), but never reaches the 5k-r=256 level (0.710). Pushing the mix to 80% LLaVA *worsens* MMBench at every rank, indicating the regression is router over-adaptation to *any* narrow training distribution, not insufficient visual exposure.

γ-free as the opposite extreme. The γ-free control (row 2) achieves the same LAMBADA gain (+4.4pp) as the canonical with slightly weaker HellaSwag (+0.6pp vs. +1.6pp) and slightly stronger MMBench (0.720 vs. 0.710). It reaches these numbers by treating the adapter as a learnable correction *on top of* the original residual path—the model still computes $x_n \approx h_{n-1} + \text{adapter}(\cdot)$, never structurally AttnRes. Because the paper's claim is that pretrained transformers can be rewired *into* the AttnRes form, we take the $\gamma \to 1$ canonical as our headline and report this row as a non-pure-AttnRes control showing the same gain is achievable with the standard residual path left in place.

A.11 Per-block skip importance

Running the 5k Qwen3-VL-2B retrofit (H_r256_5k) with a single block statically removed ($h_n \leftarrow x_n$) and measuring LAMBADA at n=300: block 1 is catastrophic (-55% acc), block 10 severe (-46%), blocks 4, 6, 11 are safest (-11 to -14%). We use this sweep to pick $\mathcal{P}=\{4, 6, 11\}$ for the dynamic-skip eligibility set. The same ranking structurally reproduces across the 340M from-scratch (Section 4.2) and the 2B retrofit: the block nearest the embedding and the one late-layer block that concentrates residual-refinement traffic are the worst to remove, while the three mid-depth positions with collapsed-onto-predecessor α are the safest.

A.12 Calibration set (dynamic skip)

Calibration uses 32 held-out LAMBADA prefixes (truncated to 512 tokens each). Per-block $w_{\text{recent}}(n)$ samples are collected with a single retrofit forward and the empirical quantile at q is used as τ_n. We verified that varying the calibration draw shifts τ_n by < 0.01 in absolute terms at the chosen q=0.95. For VLA deployment, the identical calibration pipeline applied to the VLA in-backbone forward (`retrofit/eval/calibrate_vla_thresholds.py`) produces thresholds byte-identical to the LAMBADA-calibrated set on matched inputs, confirming that the VLA and VLM forwards instantiate the same router.

A.13 Data-mix ablation (v1 → v2 → v3)

Table 18 collects the data-mix ablation that motivates the v3 canonical. Three intermediate cells show how the mix landscape partitions: v1 (narrow VL: UltraChat + LLaVA-Instruct-VSFT) is the initial canonical and preserves base within noise at 5k steps but *collapses* when trained longer; v2 (aggressive math-CoT: 40% Numina-Math/OpenThoughts/OpenMath2 $+30\%$ VL) recovers MMStar reasoning subtasks on 2B but *crashes AI2D by* 45*pp* (on 2B) and the global VL on 4B; v3 (LLaVA-OneVision-anchored) removes both failure modes and is strictly positive on VL at both scales.

Table 18: **Data-mix ablation.** Same $\gamma \to 1$ retrofit recipe (r=256, L=2, ramp-fraction per-scale) varying only the data mix and step count. v1 at 5k is preserved, but at 10k it collapses on 4B (AI2D -23pp). v2 crashes diagram-reasoning on 2B despite raising MMStar math subtasks. v3 at 10k is net-positive on every VL benchmark at 2B and matches base at 4B (with the L=4 partition of Section 6.2.1, 4B $\times$ v3 is strictly positive). All numbers from `lmms-eval` full splits.

Scale	Mix (steps)	AI2D	MMBench	MMMU	MMStar	OCRBench	RealWorldQA
2B	base	0.736	75.77	0.414	0.536	0.772	0.648
2B	v1 (5k)	0.684	72.85	0.406	0.386	0.795	0.447
2B	v1 (10k)	0.677	73.71	0.404	0.471	0.801	0.642
2B	v2 (5k)	**0.283**	73.80	0.421	0.422	0.806	0.512
2B	**v3** (10k)	**0.765**	**79.30**	**0.439**	0.534	**0.809**	**0.668**
4B	base	0.819	83.33	0.490	0.624	0.819	0.715
4B	v1 (5k)	0.810	83.76	0.510	0.579	0.812	0.707
4B	v1 (10k)	**0.580**	81.79	0.432	0.437	0.812	0.689
4B	v2 (5k)	0.603	81.87	0.477	**0.333**	0.808	0.686
4B	**v3** (10k, L=2)	0.816	**84.28**	**0.523**	0.587	0.813	0.708

The table crystallises three working rules for the retrofit's data mix. *(i) Anchor VL at* $\geq 60\%$. Dropping VL below 30% (v2) catastrophically breaks diagram reasoning. Holding VL at 50% (v1) works at 5k but not at 10k. Holding VL at 60% with a rich OneVision anchor (v3) works at 10k at both scales. *(ii) Cap math-CoT text at* $\leq 20\%$. v2's 40% math-CoT share on 2B dropped AI2D to 0.283—the retrofit's router learnt to route away from vision-heavy paths when the text side of the mix presented a strong symbolic-reasoning gradient. v3 keeps math-CoT at 20% and recovers AI2D to 0.765. *(iii) More steps does not rescue a narrow mix.* v1 at 10k crashes AI2D by 23pp on 4B while barely moving 2B, disproving the hypothesis that the v1→v2 transition was just under-training. The mix quality is binding.